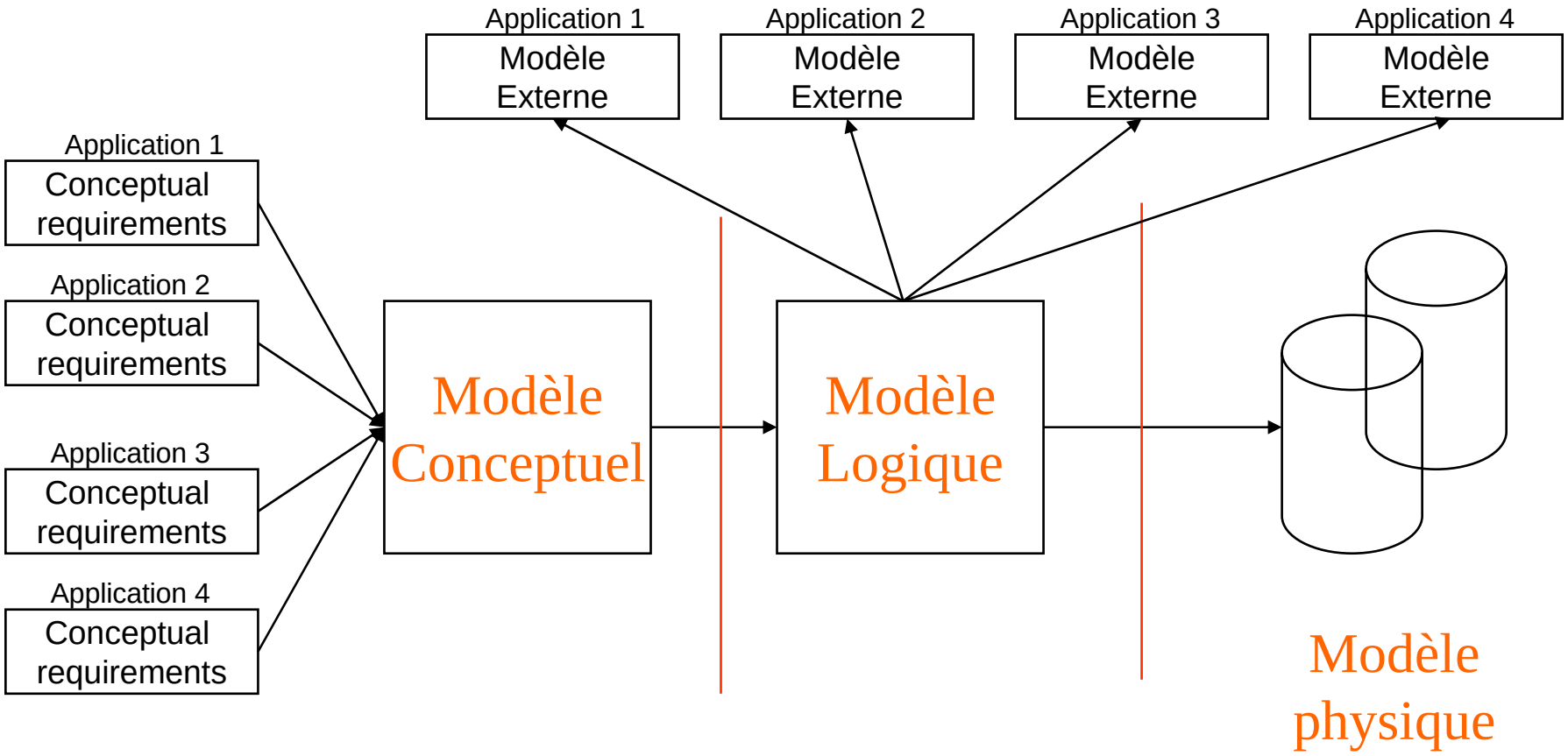
Structures d'Accès aux Données & Techniques d'Optimisation de Requêtes Relationnelles

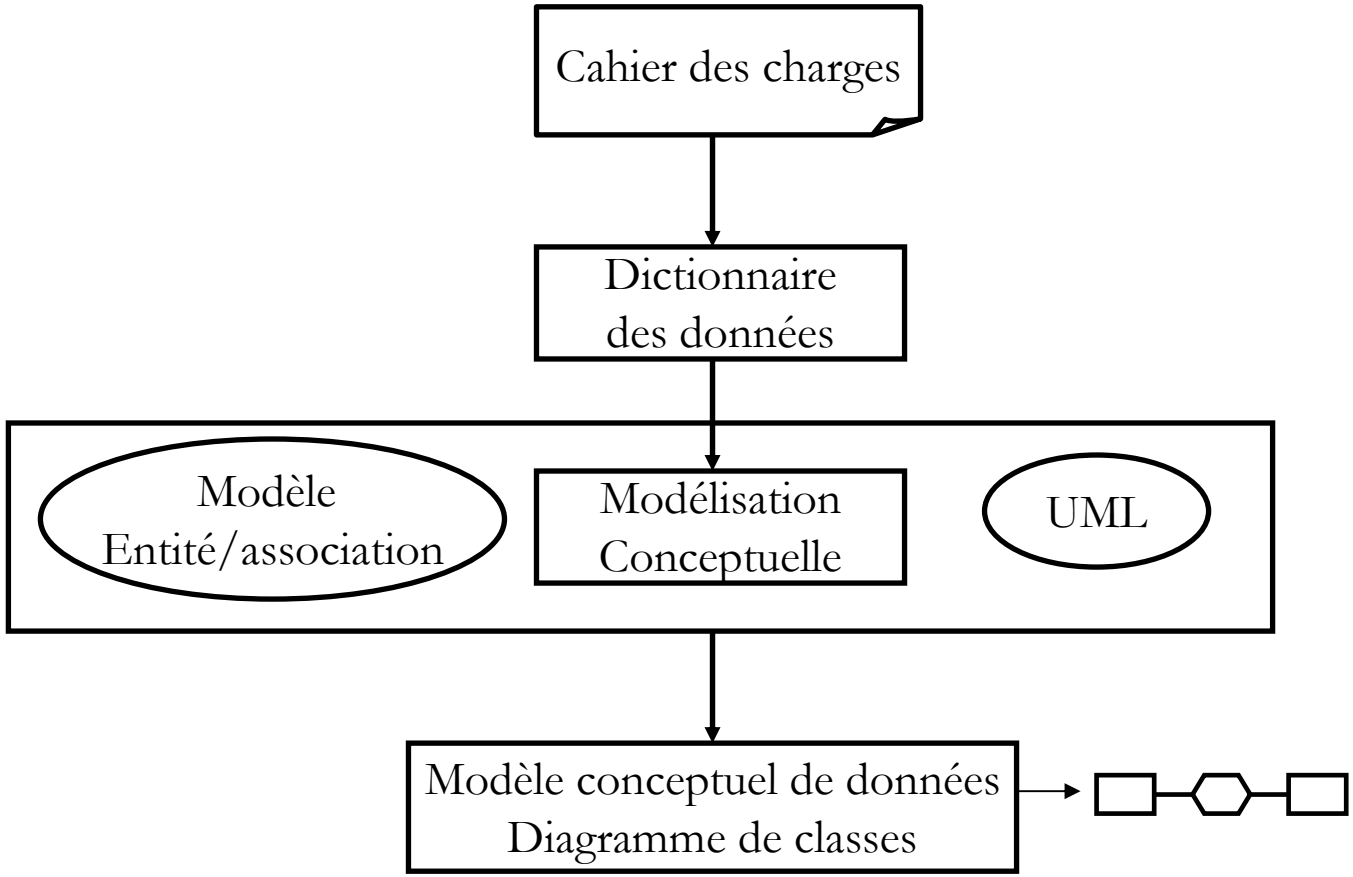
Ladjel BELLATRECHE

bellatreche@ensma.fr
05 49 49 80 72

Processus de conception d'une base de données



Modèle conceptuel



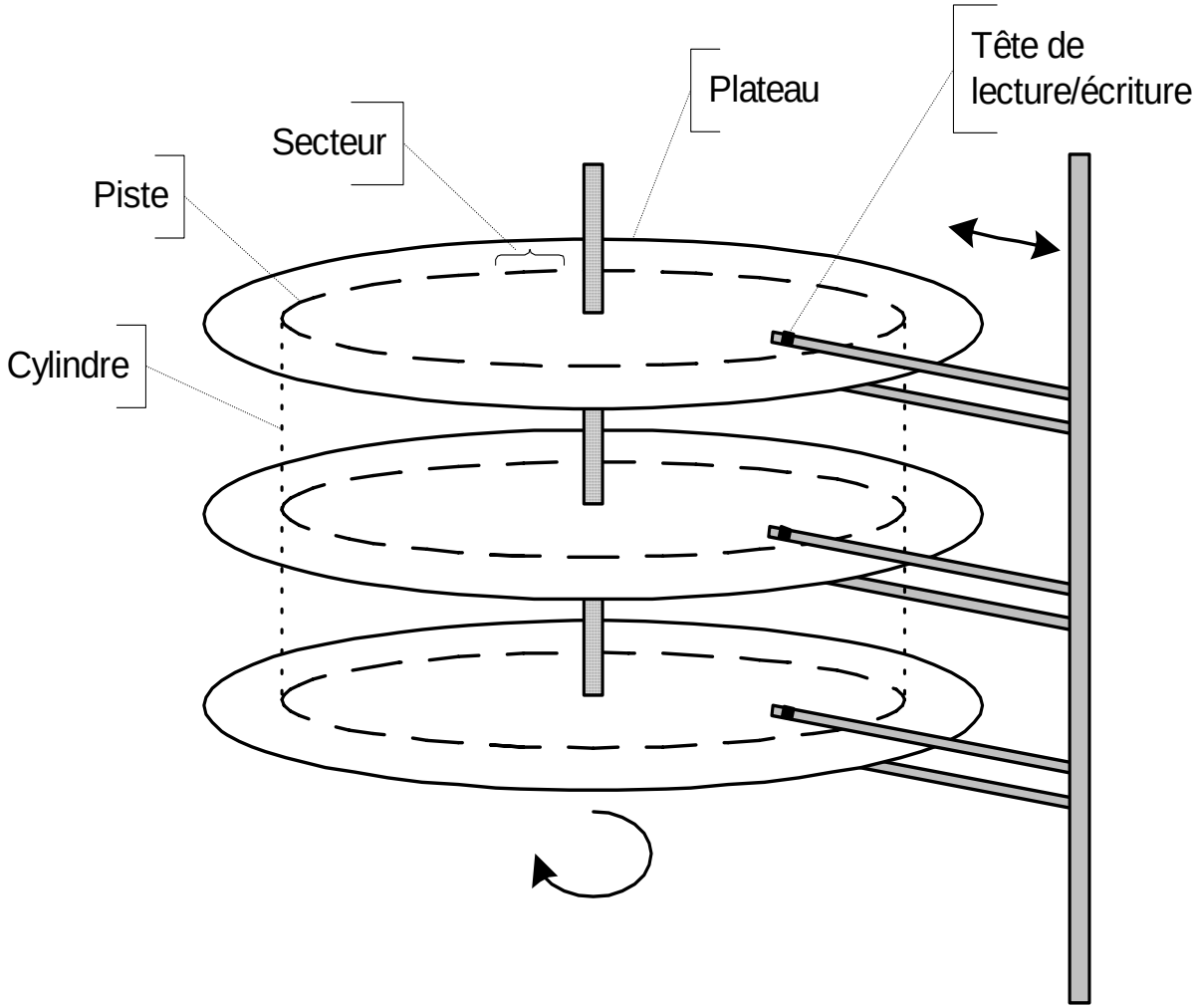
Modèle logique

- Passage du modèle conceptuel de données vers le modèle relationnel ou relationnel objet
  - Les deux règles: père - fils, père - père
- Normalisation
  - 1FN, 2FN, 3FN, etc.
    - ➔ Tables normalisées

Schéma interne

- Définit les chemins d'accès aux données
- Hypothèses générales :
  - BD est grande
  - Stockée sur le disque
- Problèmes
  - Allocation d'espace disque
  - Accès direct aux enregistrement
  - Tampons Mémoire

Structure d'un disque



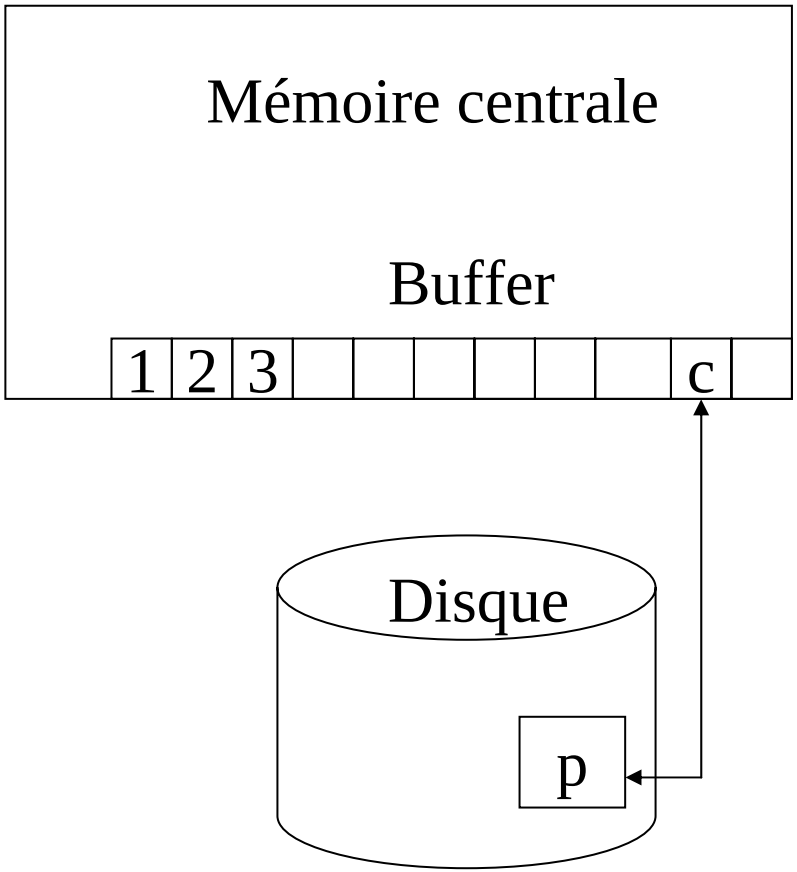
Fichiers

- Physiquement les instances d'une base de données sont stockées dans un ou plusieurs fichiers (qui peuvent être répartis)
- Un fichier est stocké sur un disque
- Il est caractérisé par un nom et une suite de blocs (pages)
- Les instances sont rangées dans des pages

Echange disque – mémoire centrale

- Echanges entre disque et mémoire centrale se font à travers d'une zone mémoire appelée tampon (buffer)
- Prise en charge de ces échanges est gérée par le gestionnaire de tampon

Structure d'un tampon



Un tampon est formé d'une suite de cases et chacune peut contenir une page

Recherche d'une page

1. Si la page *p* est dans la case *c* du tampon, retourner c (économie un accès disque)
2. Si la page *p* n'est dans la tampon, il faut la lire sur le disque. On teste s'il existe une case libre pour la recevoir. Si oui, soit *c* cette case
3. Sinon, il faut libérer une case et donc renvoyer une page du tampon sur le disque
   - LRU, FIFO
4. Si la page à rejeter a été modifiée pendant son séjour en mémoire, il faut la réécrire sur le disque.
5. Transférer la page *p* du disque dans la case *c* et retourner c.

Conception physique

- Le but : le **traitement efficace** des données (une BD est censée contenir des milliers voire des millions d'enregistrements)
- Optimiser les performances des services de base de données: demand speedy access to data, **no matter how complex the queries are**
- La conception physique nécessite des informations manipulées hors de la modélisation conceptuelle et logique

Informations nécessaires pour la conception physique

- Relations normalisées et leurs tailles estimées
- Définitions de chaque attribut
- Types d'opérations sur les données et leurs fréquences
  - interrogation, suppression, modification, insertion
- Contraintes: temps de réponse, stockage, sécurité, reprise après les pannes, contraintes d'intégrité, etc.
- Descriptions de la technologie utilisée pour l'implémentation de la base de données (Oracle, SQL Server, etc.)

Plan

- Classification des requêtes
- Méthodes d'accès
  - Techniques d'indexation
- Optimisation des requêtes
  - Expressions algébriques
  - Opérations algébriques

Typologie des requêtes (I)

- Exemple d'une BD:

  Ouvrier (NSS, Nom, Salaire, Spec, #N°U)

  Usine (N°U, Site, Prod, #NSSChef)

  Contrat(N°Cont, Fin, Cout, #NSS, #N°U)

- Classification des requêtes: [Shasha et al. ``Database tuning: A principled approach'', Prentice Hall, 1992]
  - Requête singulière (Point query)
    - Requête basée sur une seule relation et dont le prédicat de sélection est composé avec une ou plusieurs égalités de manière à ce que la réponse comprenne au plus un tuple.
  - Exemple:

    SELECT * FROM Ouvrier WHERE NSS = 1233333;

Typologie des requêtes (II)

- Requête singulière multiple (multipoint query)
  - Requête basée sur une seule relation dont le prédicat de sélection est composé d'une ou plusieurs égalités, et dont la réponse comprend plusieurs tuples
  - Exemple:

    SELECT * FROM Ouvrier WHERE Salaire > 1500 AND Spec =`soudeur ';

- Requête d'intervalle (range query)
  - Requête basée sur une seule relation et dont le prédicat comprend un test d'intervalle.
  - Exemple:

    SELECT * FROM Ouvrier WHERE Salaire > 1500 AND Salaire < 5000;

Typologie des requêtes (III)

- Requête d'appartenance à un ensemble ordonné d'attributs Z (prefix match query)
  - Requête dont les conditions d'égalités doivent inclure obligatoirement une sous-chaîne définie selon l'ensemble Z
  - Exemple: Z = {NSS, Nom}

    SELECT Nom, Salaire FROM Ouvrier WHERE NSS = 12345 AND Nom = `Dupont';

- Requête min-max (Extermal query)
  - Requête formulée sur une seule relation et dont le prédicat est composé d'égalités faisant référence à une valeur minimale ou maximale.
  - Exemple: SELECT Nom, Salaire FROM Ouvrier WHERE Salaire = max (SELECT salaire FROM Ouvrier);

Typologie des requêtes (IV)

- Requête de groupement (Grouping query)
  - Requête dont la réponse est partitionnée par un ou plusieurs attributs
  - Exemple:

    SELECT N°U, AVG(Salaire) FROM Ouvrier GROUP BY N°U;

- Requête de jointure (join query)
  - Requête basée sur plusieurs relations.
  - Exemple:

    SELECT NSS, Nom, Site FROM Ouvrier O, Usine U WHERE O.N°U = U.N°U;

Index

- Les indexes sont des fichiers qui contiennent les paires

  attribut(s) d'accès -> numéro de page

- Index primaire est construit sur la clé primaire d'une relation
- Index secondaire est construit sur un attribut quelconque d'une relation

Techniques d'indexation

- Arbre B (B-tree)
- Hachage
- Bitmap

Index

- Une fois les tuples stockés dans un fichier (disque), comment faites vous la recherche d'une manière efficace? (eg., ssn=125?)

STUDENT		
Ssn	**Name**	**Address**
123	**smith**	**main str**
234	**jones**	**forbes ave**
125	**tomson**	**main str**

1. Accès séquentiel
2. Accès direct : utilisation d'index

Index – idée principale

Index

123
125
234

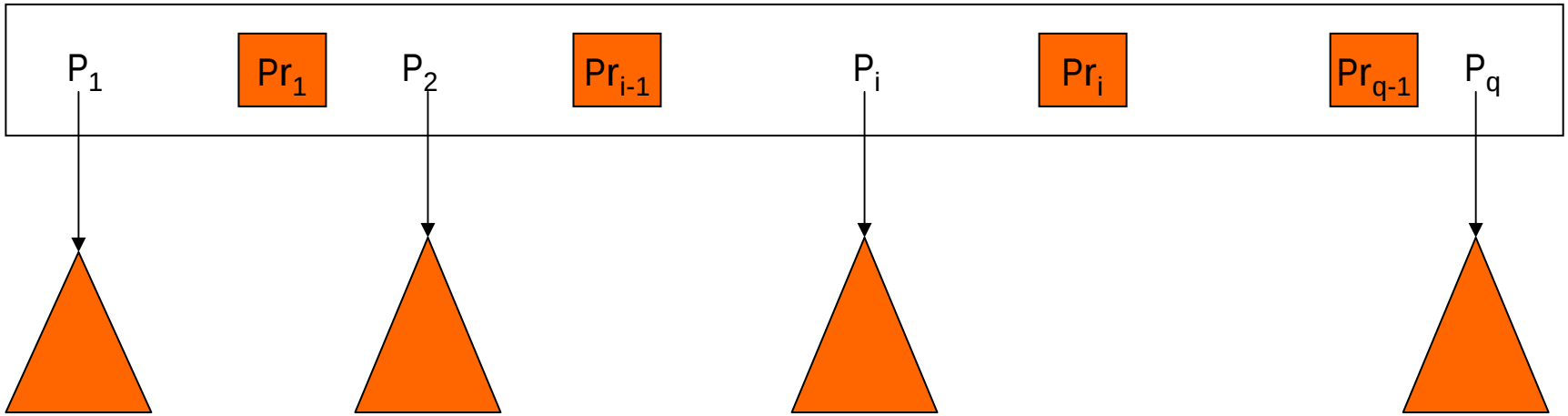
STUDENT

Ssn	Name	Address
123	smith	main str
234	jones	forbes ave
125	tomson	main str

Arbre B

- Un arbre équilibré
- Chaque nœud est un index local
- Il se réorganise dynamiquement
- Utilisé par tous les SGBDs commerciaux

Nœud d'un arbre B

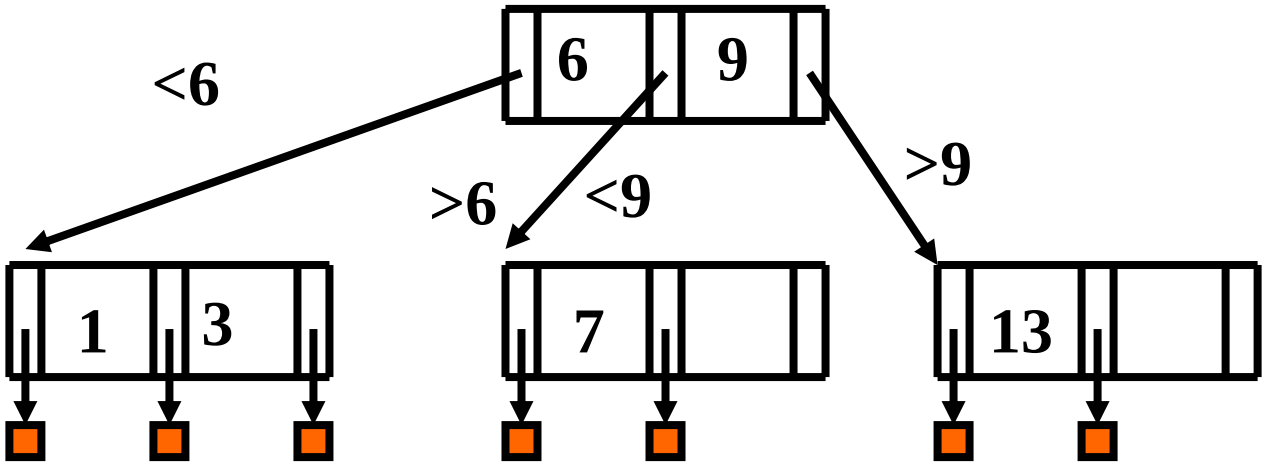


Sous-arbre pointé par P1

<P1, <Pr1>, P2, <Pr2>, ..., <Prq-1>, Pq>

- $q \leq p$
- Chaque Pi est un pointeur vers un sous-arbre
- Pri est un pointeur sur les données
- Chaque nœud a au plus p pointeurs d'arbre

Exemple

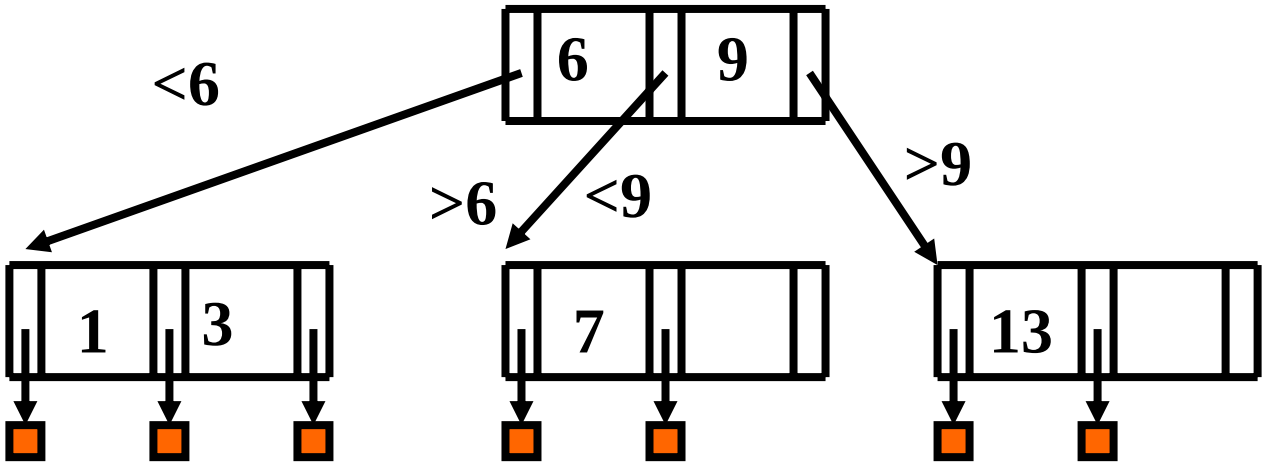


Properties

- “block aware” nodes: each node -> disk page
- O(log (N)) for everything! (ins/del/search)
- Utilization >= 50%, guaranteed; on average 69%

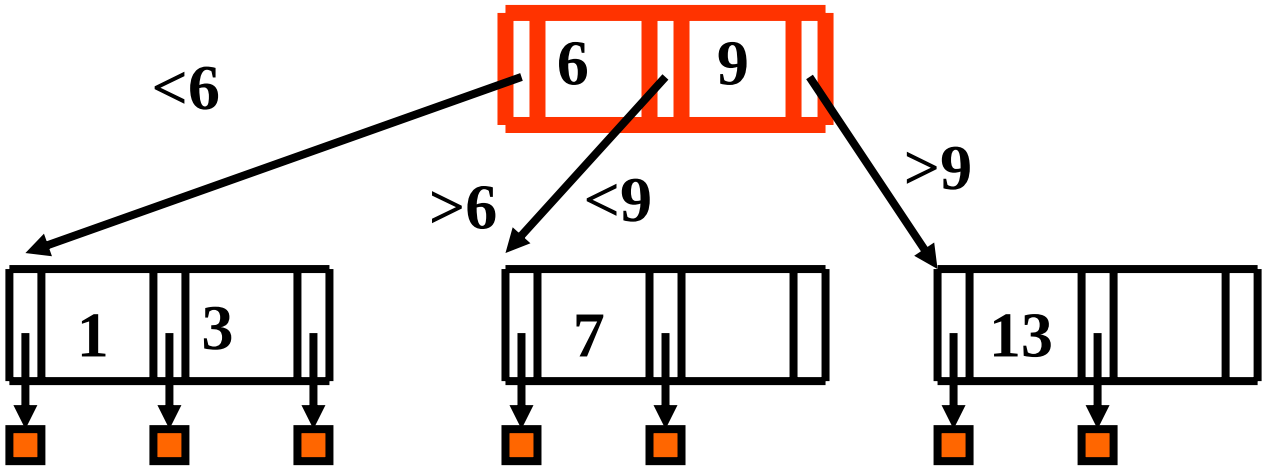
Requêtes

- Scénario pour les requêtes singulières
  - Prédicat de sélection: SSN=8



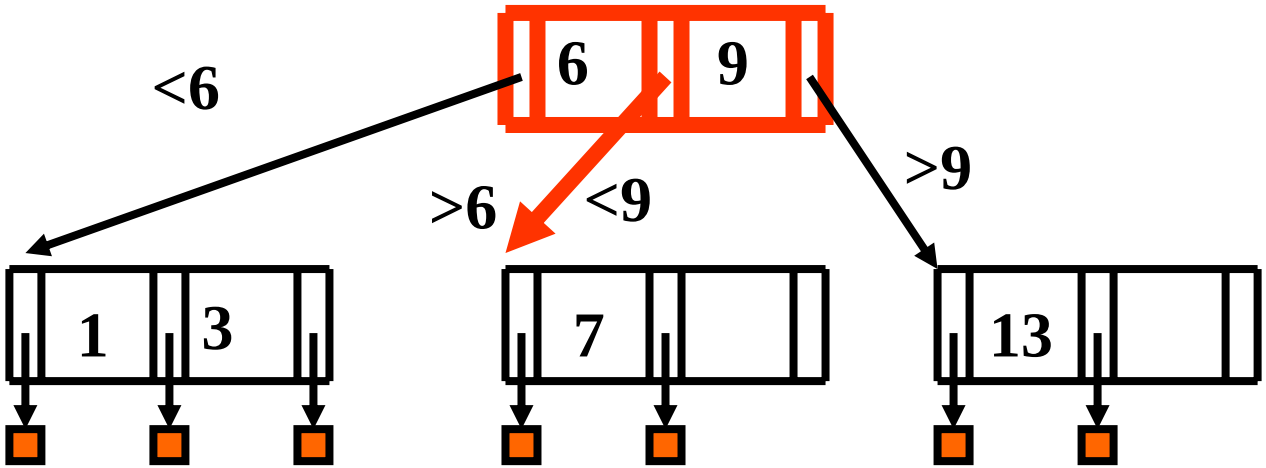
Requêtes

- Scénario pour les requêtes singulières
  - Prédicat de sélection: SSN=8



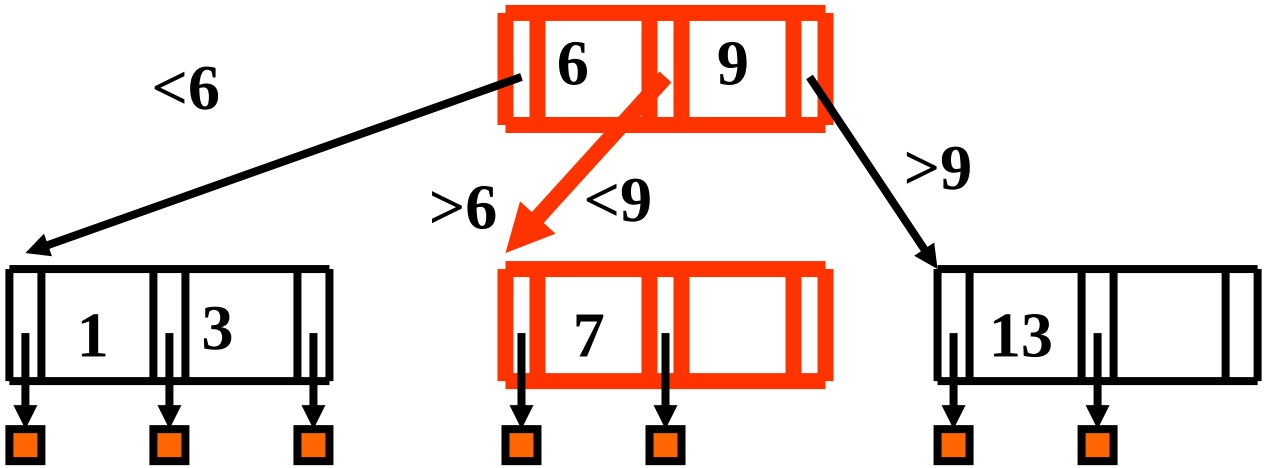
Requêtes

- Scénario pour les requêtes singulières
  - Prédicat de sélection: SSN=8



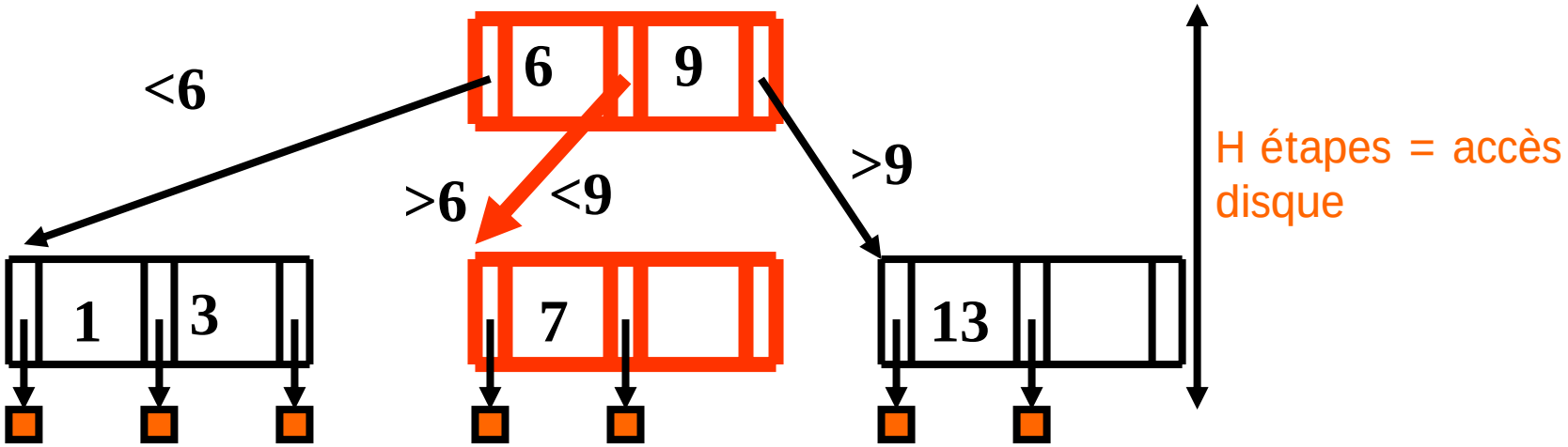
Requêtes

- Scénario pour les requêtes singulières
  - Prédicat de sélection: SSN=8



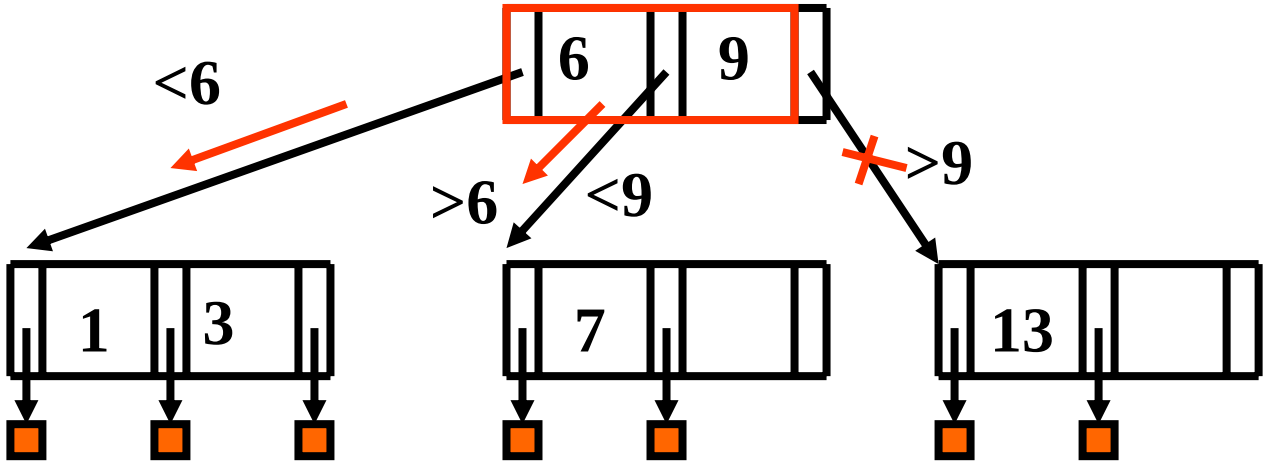
Requêtes

- Algorithme d'accès



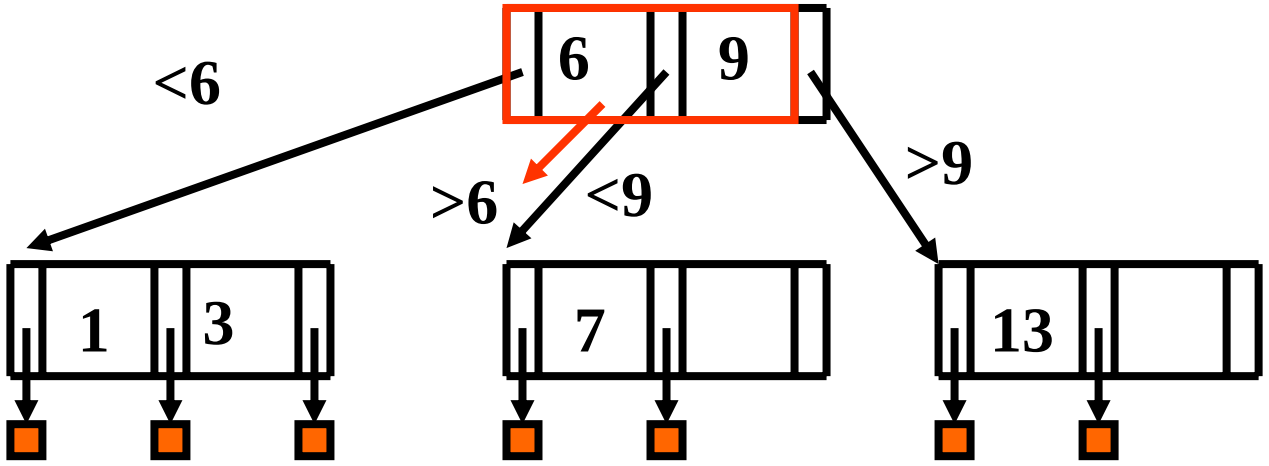
Requêtes d'intervalle

- Exemple: 5<salaire<8
- Proximity/ nearest neighbor searches? (eg., *salaire ~ 8*)



Requêtes d'intervalle

- Exemple: 5<salaire<8
- Proximity/ nearest neighbor searches? (eg., *salaire ~ 8*)



Recherche dans une table avec un index

- Exemple: SELECT * FROM Etudiant WHERE Age = 24;
- Un index sur Age permet d'obtenir rapidement les RIDs des tuples des employés de 24 ans.
- Requêtes conjonctives

  SELECT * FROM Etudiant WHERE Salaire = 2000 AND Ville = `Poitiers ';

  - Supposons l'existence de deux index; un sur Salaire et l'autre sur Ville.
  - Le premier index fournit les RIDs satisfaisant la 1ère condition qui seront regroupés dans une table temporaire T1. Idem pour le deuxième index ~ T2
  - Enfin l'intersection de T1 et T2 donne la réponse

Requêtes conjonctives ~ Index

- Attention:
  - Cette solution pourrait être inefficace si les facteurs de sélectivité des prédicats sont supérieurs à 0.6
- Dans un tel cas, le SGBD pourrait préférer faire un balayage séquentiel de la table

Index bitmap

- Problème:
  - Comment indexer une table sur un attribut qui ne prend qu'un petit nombre de valeurs?

Principe de l'index bitmap

- Soit un attribut A, prenant n valeurs possibles [v_1, ...,v_n] (domaine)
- Création d'un index bitmap sur l'attribut A:
  - On crée n tableaux de bit, un pour chaque valeur v_i
  - Le tableau contient un bit pour chaque tuple t
  - Le bit d'un tuple t est à 1 si: t.A = v_i, à 0 sinon

Exemple

ROWID(RID)	Soudeur	Fraiseur	Sableur	Tourneur
00055 :000 :0023	0	1	0	0
00234 :020 :8922	1	0	0	0
19000 :328 :6200	0	0	0	1
21088 :120 :1002	0	0	1	0

Ouvrier(NSS, Nom, Salaire, Spécialité, N°U)
avec le domaine de l'attribut Spécialité est: {Soudeur, Fraiseur, Sableur, Tourneur}

SQL: CREATE INDEX BITMAP ON Ouvrier(Specialite) TABLESPACE spec_espace

Intérêt de Bitmap: Recherche

- Soit la requête suivante:

  SELECT * FROM Ouvrier WHERE Spec = ''Soudeur''

  - On prend le tableau pour la valeur Soudeur
  - On garde toutes les cellules à 1
  - On accède aux enregistrement par l'adresse

- ➔ Très efficace si le nombre de valeurs (n) est petit
  - Exemple: Sexe

Autre exemple

SQL: CREATE INDEX BITMAP ON Ouvrier(Salaire) TABLESPACE Sal_espace

ROWID(RID)	20000	30000	40000
00055 :000 :0023	1	0	0
00234 :020 :8922	0	1	0
19000 :328 :6200	0	0	0
21088 :120 :1002	0	0	1

Requête: Quels sont les employés soudeurs qui gagnent 30 000

Réponse: RID: 00234:020:8922

Quelle est la procédure?

Exemple avec count()

- Requêtes type OLAP (data warehouse)

  SELECT COUNT(*) FROM Ouvrier WHERE Spec in (‘Soudeur’, ‘Tourneur’)

- On compte le nombre 1 dans la colonne Soudeur
- On compte le nombre 1 dans la colonne Tourneur
- On fait la somme et c’est terminé

Principe du hachage

- Calcul la position d'un enregistrement d'après le clé
  - une fonction de hachage h associe des valeurs de clés à des adresses de bloc
  - recherche d'un enregistrement implique le calcul de l'endroit où il se trouve

Le hachage : principe

- Hachage ou adressage dispersé (hashing)
- Fichier = tableau de *paquets* (***bucket***)
  - ~ ARRAY paquet [0..*TH*-1],
    - *TH* : *taille de l'espace d'adressage primaire*
- Fonction *h(clé de hachage)* ➔ l'adresse d'un **paquet**
- Habituellement paquet = bloc(page)
- Pas d'index à traverser : ***O***(1) en meilleur cas
- Sélection par égalité

Exemple

Bloc (page) = 4 enregistrements

h(clé) = clé MOD 3

clé = 10

$h(10) = 10 \text{ MOD } 3 = 1$

0	60	Erable argenté	15.99
	90	Pommier	25.99
	81	Catalpa	25.99
1	70	Herbe à puce	10.99
	40	Epinette bleue	25.99
	10	Cèdre en boule	10.99
2	20	Sapin	12.99
	50	Chêne	22.99
	95	Génévrier	15.99
	80	Poirier	26.99

Recherches et mises à jour

- Recherches :
  - Efficace pour les requêtes singulières
  - Requêtes par préfixe (LIKE)
  - Inefficace pour les requêtes d'intervalle
- Mises à jour :
  - On ne peut pas changer un enregistrement de place
  - Création de chaînage de pages quand une page déborde

➔ **Performances se dégradent**

Exemple de mise à jour

0	60	Erable argenté	15.99
	90	Pommier	25.99
	81	Catalpa	25.99
1	70	Herbe à puce	10.99
	40	Epinette bleue	25.99
	10	Cèdre en boule	10.99
2	20	Sapin	12.99
	50	Chêne	22.99
	95	Génévrier	15.99
	80	Poirier	26.99

Instance à insérer

89	Citronnier	29.45

89	Citronnier	29.45

Hachage vs indexage

- **O**(1) en meilleur cas vs **O**(log(N))
- Pas d'espace supplémentaire d'index
- Gaspillage d'espace si *TH* trop grand
- Performance dégradée si TH trop petit
- Gestion plus délicate
  - La détermination de la fonction de hachage h et TH
  - Maintenance : réorganisations

Les choix d'Oracle

- Par défaut l'index est un arbre B
  - Dès qu'on utilise une commande PRIMARY KEY, Oracle crée un arbre B sur la clé primaire
  - Arbre est stocké dans un segment d'index
- Index bitmap
- Hachage

Exemples de création d'index

- Le nom d'un index est choisi par le DBA et peut être normalisé en le préfixant par idx_ suivi du nom de l'attribut indexé et terminé par le nom de la table
- Exemple:

```
CREATE UNIQUE INDEX idx_NSS_Ouvrier ON Ouvrier(NSS);
CREATE UNIQUE INDEX idx_NSS_Salaire_Ouvrier ON Ouvrier(NSS, Salaire);
CREATE INDEX BITMAP ON Ouvrier(Specialite) TABLESPACE spec_espace
```

- Suppression d'un index

```
DROP INDEX <Nom de l 'index>;
DROP INDEX idx_NSS_Ouvrier;
```

Guide d'utilisation des index en cours d'exploitation d'une BD

- Les index utiles sont ceux nécessaires à l'exploitation courante des données
- Supprimer les index lorsqu'il s'agit de traiter un flux important de transactions de type mise à jour sur une table. Ils sont d'abord supprimés, puis recréés au besoin après les mises à jour
- Une accélération de la jointure est possible par la création des index: indexer les attributs de jointure permettra un accès plus rapide

Guide d'utilisation des index en cours d'exploitation d'une BD (suite)

- Tous les attributs d'un index composé doivent être présents dans la clause WHERE, peu importe leur ordre dans la clause, pour que l'optimiseur en tire profit
- Si le premier attribut de l'index composé est présent dans le WHERE, l'index composé est utilisé. Sinon, il devient inutile
- La négation dans une condition bloque l'usage des index composés: les valeurs hors domaine ne sont pas indexées
- Quand faut-créer un index composé?
  - Lorsque certains attributs sont fréquemment utilisés ensemble, il peut être avantageux de définir quelques index composés

Surveillance de l'utilisation des index

- Oracle (à partir du 9i) permet de surveiller les index afin de déterminer s'ils sont utilisés ou no.
- SQL :

  ALTER INDEX nom_index MONITORING USAGE |

  NOMONITORING USAGE

  - permet d'activer ou de désactiver la surveillance

- ALTER INDEX idx_NSS_Ouvrier MONITORING USAGE;

Mise en pratique

SELECT *

FROM V$OBJECT_USAGE % une vue permettant la surveillance des index (6 attributs)

WHERE index_name = 'idx_NSS_Ouvrier ';

INDEX_NAME	TABLE_NAME	MONITORING	USED	START_MONITORING	END_MONITORING
idx_NSS_Ouvrier	Ouvrier	**YES**	**YES**	08/10/2005 16:15:00	

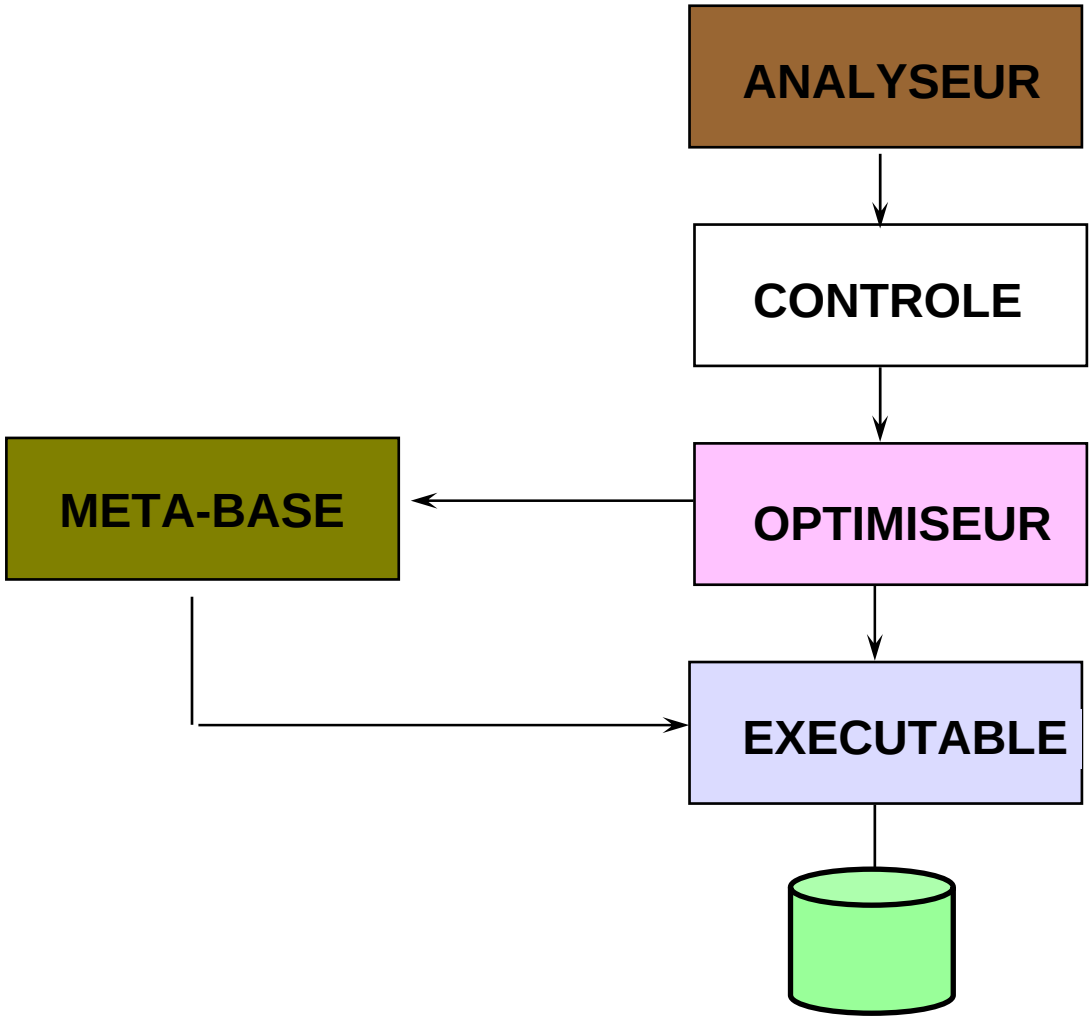
➔ Cet index est utilisé au moins une fois

INDEX_NAME	TABLE_NAME	MONITORING	USED	START_MONITORING	END_MONITORING
idx_NSS_Ouvrier	Ouvrier	**YES**	**NO**	08/10/2005 16:15:00	

➔ DROP INDEX idx_NSS_Ouvrier;

Optimisation de requêtes

Introduction



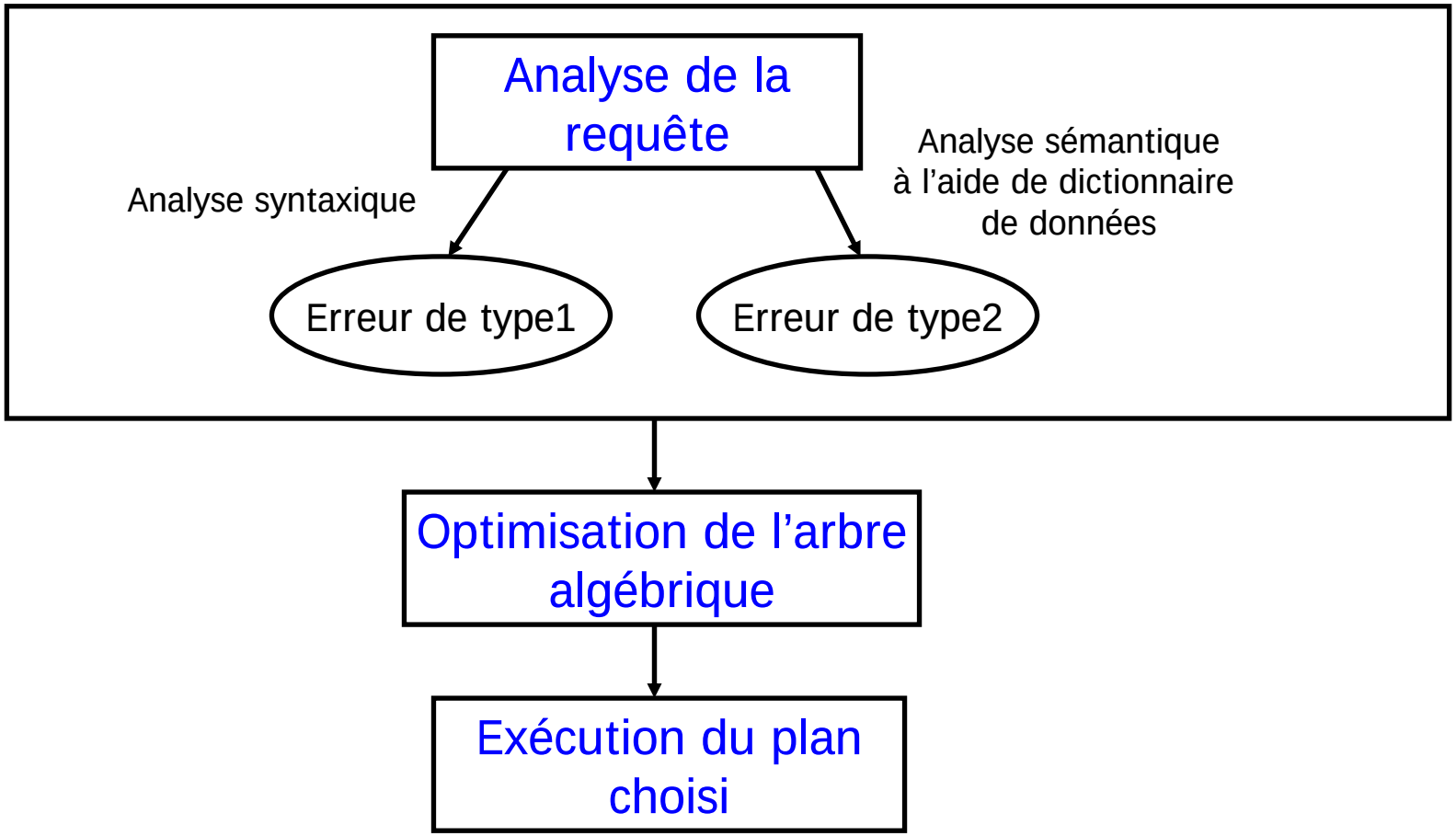
SYNTAXE
SEMANTIQUE
SCHEMA

VUES
INTEGRITE
AUTORISATIONS

ORDONNANCEMENT
ELABORATION
D'UN PLAN

EXECUTION
METHODES D'ACCES

Traitement d'une requête SQL



Arbre algébrique

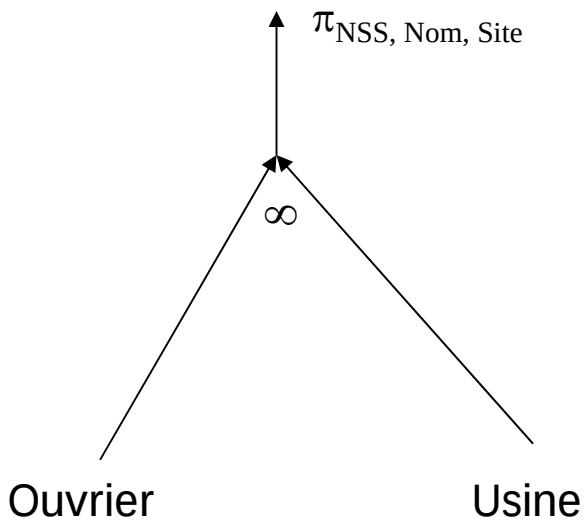
- **algèbre relationnelle** : modèle formel permettant d'exprimer et de calculer les requêtes sur les relations.
  - Constituée d'un ensemble d'opérateurs algébriques. Les plus utilisés sont la sélection, la jointure, la projection et les opérateurs ensemblistes.
  - Le résultat de l'exécution d'un opérateur est toujours une relation, ce qui permet la composition.
  - Utilisée également pour l'optimisation de requêtes.
  - Une requête algébrique peut se présenter sous forme d'un arbre algébrique.
  - Arbre algébrique : visualisation d'une requête sous une forme graphique d'arbre plus facile à lire qu'une forme linéaire.

Arbre algébrique

Un arbre algébrique est un arbre représentant une requête avec:

- **Les nœuds feuilles sont les relations de base**
- **Les nœuds intermédiaires sont les opérateurs**
- **Le nœud racine est le résultat**
- **L'arc est un flux de données**

Exemple: SELECT NSS, Nom, Site FROM Ouvrier O, Usine U WHERE O.N°U = U.N°U;

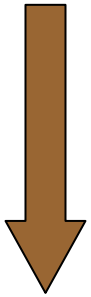


Techniques d'optimisation

- Expressions algébriques:
  - Transformation des requêtes pour obtenir la meilleure séquence d'opérations
    - Approche qui marche bien (mais pas toujours)
- Opérations algébriques:
  - Coût des différentes stratégiques possibles en fonction des caractéristiques des fichiers sur lesquels sont implantées les relations
- Généralement les SGBDs commerciaux combinent les deux stratégies

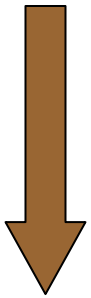
Technique basée sur expressions algébriques

Requête (SQL)



Traduction

Requête en algèbre

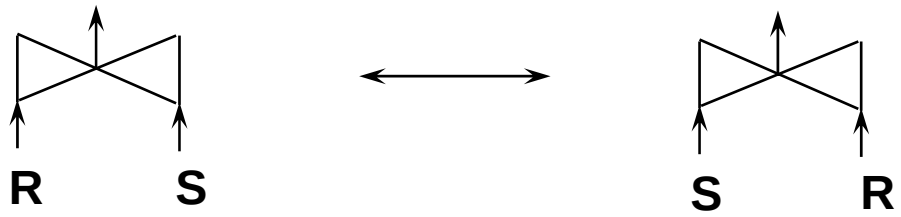


Transformation en fonction des propriétés des opérateurs

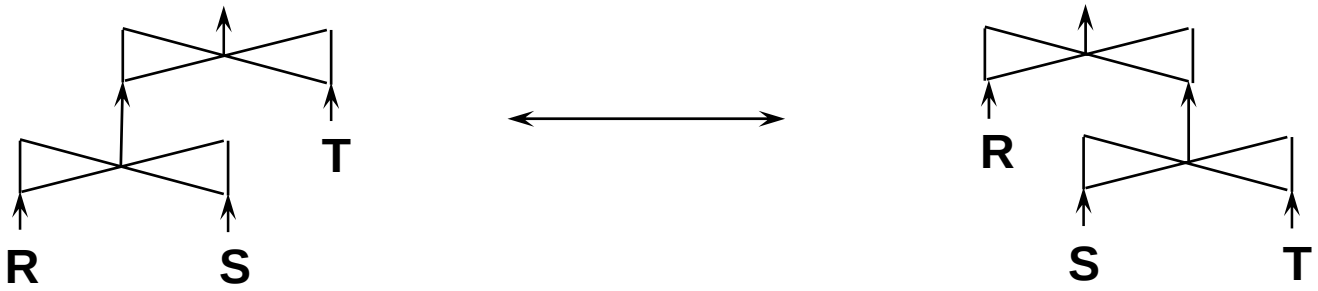
Plan optimal

Propriétés des opérateurs

- Commutativité pour jointure et produit



- Associativité pour jointure et produit



➔ Il existe N!/2 arbres de jointure de N relations.

➔ Parmi les jointures, certaines sont des produits cartésiens.

Propriétés des opérations (suite)

- Groupage des projections

  $\pi_{A1, ..., An} (\pi_{B1, ..., Bm}(E)) = \pi_{A1, ..., An} (E)$ --- $\{A_1, ..., A_n\}$ inclus dans $\{B_1, ..., B_m\}$

- Exemple
  - $\pi_{nom, prénom}(\pi_{[age}$ (Étudiant))?
  - $\pi_{nom, prénom}(\pi_{nom, prénom, age}$ (Étudiant))

- Groupages de sélections

  $\sigma_{F1} (\sigma_{F2} (E)) = \sigma_{F1 \wedge F2}(E)$

- Exemple
  - $\sigma_{age>20} (\sigma_{sexe='F'}$ (Étudiant)) $= \sigma_{(age>20) \wedge (sexe='F')}$(Étudiant)

- Inversion σ et π
  - $\pi_{A1, ..., An}(\sigma_F(E)) = \sigma_F(\pi_{A1, ..., An}(E))$ si F porte sur $A_1, ..., A_n$
  - $\pi_{A1, ..., An}(\sigma_F(E)) = \pi_{A1, ..., An} (\sigma_F(\pi_{A1, ..., An, B1, ..., Bn}) (E))$

    si F porte sur $B_1, ..., B_m$ qui ne sont pas parmi $A_1, ..., A_n$

Propriétés des opérations (suite)

- Inversion la sélection et le produit

  $\sigma_F (E_1 * E_2) = \sigma_F (E_1) * E$ si F ne porte que sur les attributs de E_1

  $\sigma_F (E_1 * E_2) = \sigma_{F1} (E_1) * \sigma_{F2} (E_2)$

  Si $F = F_1 \wedge F_2$ et F_i ne porte que sur les attributs de E_i

  $\sigma_F (E_1 * E_2) = \sigma_{F2} (\sigma_{F1} (E_1) * E_2)$ si F_1 porte sur les attributs de E_1 et F_2 sur ceux de E_1 et E_2

- Exemple
  - $\sigma_{sexe = 'F'}$(Étudiant * Cours) = $\sigma_{sexe = 'F'}$(Étudiant) * Cours
- Inversion sélection et union
  - $\sigma_F (E_1 \cup E_2) = \sigma_F(E_1) \cup \sigma_F(E_2)$
- Inversion sélection et différence
  - $\sigma_F (E_1 - E_2) = \sigma_F(E_1) - \sigma_F(E_2)$
- Inversion projection et produit
- Inversion projection et union

Principes d'optimisation des expressions algébriques

- Exécuter les sélections aussitôt que possible
- Réduire la taille des relations à manipuler
  - ➔ Combiner les sélections avec un produit cartésien pour aboutir à une jointure
- Combiner des séquences d'opérations unaires (σ, π)
  - ➔ Cascade d'opérations appliquées groupée à chaque tuple
- Mémoriser les sous-expressions communes

Exécuter les sélections aussitôt que possible

- Appliquer ces règles (remplacer membre gauche par droit)
  - $\sigma_F (E_1 \cup E_2) = \sigma_F(E_1) \cup \sigma_F(E_2)$
  - $\sigma_F (E_1 - E_2) = \sigma_F(E_1) - \sigma_F(E_2)$
  - $\sigma_F (E_1 * E_2) = \sigma_F (E_1) * E_2$ si F ne porte que sur les attributs de E_1
  - $\sigma_F (E_1 * E_2) = \sigma_{F1} (E_1) * \sigma_{F2} (E_2)$ si $F = F_1 \wedge F_2$ et F_i ne porte que sur les attributs de E_i
  - $\sigma_F (E_1 * E_2) = \sigma_{F2} (\sigma_{F1} (E_1) * E_2)$ si F_1 porte sur les attributs de E_1 et F_2 sur ceux de E_1 et E_2

Combiner les sélection avec un produit cartésien pour aboutir à une jointure

- Transformation si possible du produit cartésien en jointure:
  - Si R * S est l'argument d 'une sélection σ_F alors si F est une comparaison entre attributs de R et de S, transformer R * S en jointure : σ_F (R * S) = (R ∞_F) S
- Attention : si F est une expression ne portant que sur les attributs de R:

$$\sigma_F (R * S) = \sigma_F (R) * S$$

Combiner des séquences d'opérations unaires

- Un ensemble d'opérations unaires peuvent être combinées et appliquées ensemble sur chaque tuple manipulé

Sous-expressions communes à une expression

- Si une expression contient plusieurs fois la même sous-expression
- Si cette sous-expression peut être lue en moins de temps qu'il faut pour la calculer
  - ➔ Matérialiser la sous-expression commune
- Principe de vues matérialisées (voir le cours sur les data warehouse)

Un algorithme d'optimisation

- Séparer chaque sélection $\sigma_{F1 \wedge \ldots \wedge Fn}(E)$ en une cascade $\sigma_{F1}(\sigma_{F1} \ldots (\sigma_{Fn}(E)))$
- Descendre chaque sélection le plus bas possible dans l'arbre algébrique
- Descendre chaque projection aussi bas possible dans l'arbre algébrique
- Combiner des cascades de sélection et de projection dans une sélection seule, une projection seule, ou une sélection suivie par une projection
- Regrouper les nœuds intérieurs de l'arbre autour des opérateurs binaires (*, -, $\cup$). Chaque opérateur binaire crée un groupe
- Produire un programme comportant une étape pour chaque groupe, à évaluer dans n'importe quel ordre mais tant qu'aucun groupe ne soit évalué avant ses groupes descendants.

Exercice

- Soient trois relations: Livre, Prêt et Lecteur

  Livre(ISBN, titre, auteur, éditeur)

  Prêt(N°carteP, ISBNP, date)

  Lecteur(N°carte, nom, adresse)

- Requête: Liste des noms des lecteurs et des titres des livres pour tous les prêts effectués avant le 15/6/1990
- Trouver le meilleur plan d'exécution de cette requête

Quelques problèmes

- La restructuration d'un arbre sous-entend souvent la permutation des opérateurs binaires
- Cette optimisation ignore la taille de chaque table, les facteurs de sélectivité, etc.
- Aucune estimation de résultats intermédiaires
- Exemple:

  (Ouvrier ∞ Usine) ∞ Contrat

  - Si cardinalité(Usine) <<<Cardinalité(Contrat) << Cardinalité(Ouvrier) alors l'expression suivante a des chances d'être plus performante: (Ouvrier ∞ Contrat) ∞ Ouvrier

➔ Optimisation basée sur les modèles de coût

Méthodes d'optimisation basées sur les modèles de coûts

«Cost-based Methods»

Optimisation des opérations: Notions (I)

- Les tables relationnelles sont stockées physiquement dans des fichiers sur disque
- Lorsqu 'on veut accéder aux données, on doit transférer le contenu pertinent des fichiers dans la mémoire
  - Fichier = séquence de tuples
  - Bloc (page) = unité de transfert de données entre disque et mémoire
  - Facteur de blocage = nombre de tuples d'une relation qui «tiennent» dans un bloc
  - Coût de lecture (ou écriture) d'une relation = nombre de blocs à lire du disque vers la mémoire (ou à écrire de la mémoire vers le disque)

Estimation du coût des opérations physiques

- TempsES : temps d'accès à mémoire secondaire (MS)
- TempsUCT
  - Souvent négligeable
- TailleMC : espace mémoire centrale
- TailleMS : espace mémoire secondaire

Modèle du coût d'une entrée-sortie en mémoire secondaire

Paramètre	**Signification**
TempsESDisque(*n*)	Temps de total transfert (lecture ou écriture) de *n* octets du disque
TempsTrans(*n*)	Temps de transfert des *n* octets sans repositionnement
TempsPosDébut	Temps de positionnement au premier octet à transférer (ex : 10 *ms*)
TempsRotation	Délai de rotation (ex : 4 *ms*)
TempsDépBras	Temps de déplacement du bras (ex : 6 *ms*)
TauxTransVrac	Taux de transfert en vrac (ex : 2MB/sec)
TempsESBloc	Temps de transfert d'un bloc (ex : 11 *ms*)
TempsTrans	Temps de transfert d'un bloc sans repositionnement (ex : 1 *ms*)
TailleBloc	Taille d'un bloc (ex : 2K octets)

Statistiques au sujet des tables

Statistique	Signification
$\|\|T\|\|$	Nombre de lignes de la table T
$TailleLigne_T$	La taille d'une ligne de la table T
FB_T	Facteur de blocage moyen de T
$NDIST$	Nombre de valeurs distinctes de la colonne pour la table T
$Min_T(colonne)$	Valeur minimum de la colonne de T
$Max_T(colonne)$	Valeur maximum de la colonne de T
$\|T\|$	Nombre de pages de la table T

Facteur de sélectivité

- $||(\sigma(R))|| = SF * ||R||$ avec:
  - **SF (A = valeur) = 1 / NDIST(A)**
  - **SF(A > valeur) = (max(A) - valeur) / (max(A) - min(A))**
  - **SF(A < valeur) = (valeur - min(A)) / (max(A) - min(A))**
  - **SF(A IN liste valeurs) = (1/NDIST(A)) * CARD(liste valeurs)**
  - **SF(P et Q) = SF(P) * SF(Q)**
  - **SF(P ou Q) = SF(P) + SF(Q) - SF(P) * SF(Q)**
  - **SF(not P) = 1 - SF(P)**
- Le coût dépend de l'algorithme (index, hachage ou balayage).

Facteur de sélectivité

- La taille d'une jointure est estimée par la formule suivante:

$$||R1 \infty R2|| = Sel * ||R1|| * ||R2||;$$

avec sel: la sélectivité de la jointure

Exemple

```
SELECT *
FROM R1, R2
```

→ **SF = 1**

```
SELECT *
FROM R1
WHERE A = valeur
```

→ **SF = 1/NDIST(A) avec un modèle uniforme**

Plans d’exécutions

Ce qu'il faut savoir

- Qu'est ce qu'un plan d'exécution
  - Un programme combinant des opérateurs physiques (chemins d 'accès et traitements de données)
  - Il a la forme d'un arbre: chaque nœud est un opérateur qui
    - prend des données en entrée
    - applique un traitement
    - produit les données traitées en sortie

Algorithmes Généraux pour les opérateurs relationnels

La sélection

- Sélection sans index

  SELECT NSS, Nom FROM Ouvrier WHERE Salaire > 15000

- Plan d'exécution

  pour chaque tuple t de ouvrier faire -- accès séquentiel

  si t.salaire > 15000 alors réponse = réponse $\cup$

  $\pi_{NSS,\ Nom}$ (Ouvrier)

Algorithmes de jointure

- Jointure sans index
  - Jointure par boucles imbriquées (nested loop)
  - Jointure par tri-fusion (sort-join)
  - Jointure par hachage (hash-join)
- Jointure avec index
  - Jointure avec boucles indexées

Jointure par boucles imbriquées (JBI)

- $R \infty_F S$
  - Principe:
    - La première table est lue séquentiellement et de préférence stockée entièrement en mémoire
    - Pour chaque tuple de R, il y a une comparaison avec les tuples de S

Jointure par boucles imbriquées

```
POUR chaque ligne l_R de R
        POUR chaque ligne l_S de S
                SI θ sur l_R et l_S est satisfait
                        Produire la ligne concaténée à partir de l_R et l_S
                FINSI
        FINPOUR
FINPOUR
```

+ Simple

Complexité: (||R|| + (||R||*||S||)

Si la table extérieure tient en mémoire: une seule lecture des deux tables suffit.

Jointure par tri fusion

- Plus efficace que les boucles imbriquées pour les grosses tables
- Principe:
  - Trier le deux tables sur les colonnes de jointure
  - Effectuer la fusion
- Complexité: Le tri qui coûte cher
- Quelle est la complexité de cet algorithme?

Jointure par tri fusion

```
Trier R et S par tri externe et réécrire dans des fichiers temporaires
Lire groupe de lignes G_R(c_R) de R pour la première valeur c_R de clé de jointure
Lire groupe de lignes G_S(c_S) de S pour la première valeur c_S de clé de jointure
TANT QUE il reste des lignes de R et S à traiter
        SI c_R = c_S
                Produire les lignes concaténées pour chacune des combinaisons de
                        lignes de G_R(c_R) et G_S(c_S);
                Lire les groupes suivants G_R(c_R) de R et G_S(c_S) de S;
         SINON
                SI c_R < c_S
                        Lire le groupe suivant G_R(c_R) de R
                SINON
                        SI c_R > c_S
                                Lire le groupe G_S(c_S) suivant dans S
                        FINSI
                FINSI
        FINSI
FIN TANT QUE
```

Jointure par hachage

- Algorithme récent, encore peu répandu
- Très efficace quand une des deux tables est petite (1, 2, 3 fois la taille de la mémoire)
- Algorithme en option dans Oracle

Principe de la jointure par hachage

❶ On hache la plus petite des deux tables en n fragments

❷ On hache la seconde table, avec la même fonction, en n autres fragments

❸ On réunit les fragments par paire, et on fait la jointure

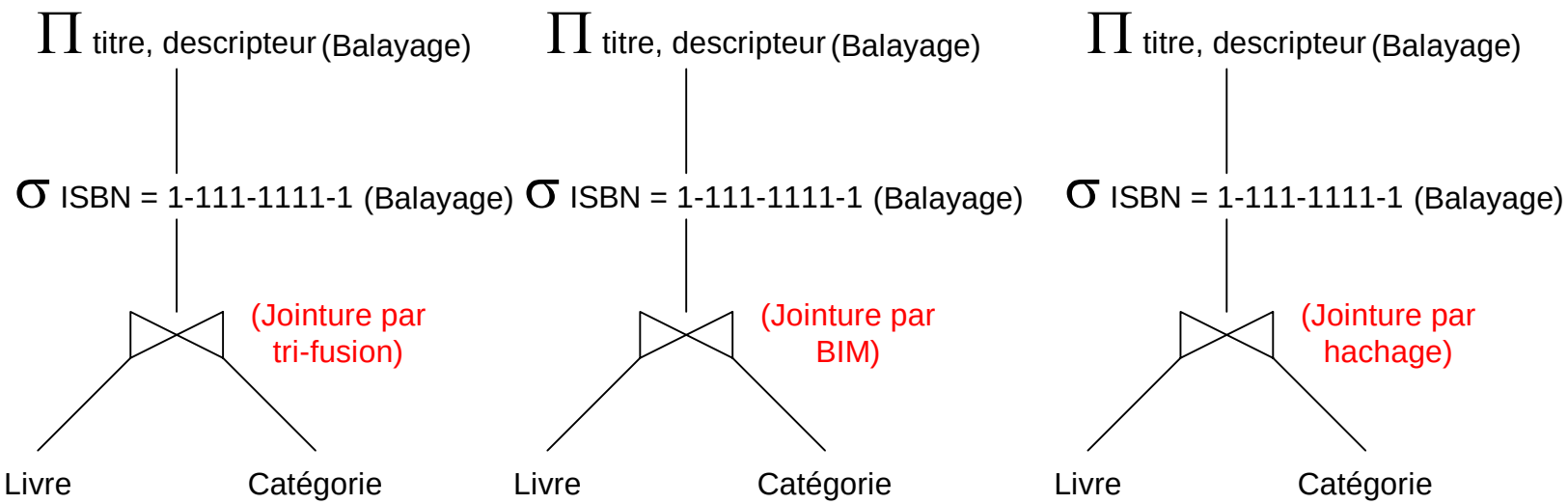
⌘ Essentiel: pour chaque paire, au moins un fragment doit tenir en mémoire

Optimisation

- Chercher le meilleur plan d 'exécution?
  - coût excessif
- Solution approchée à un coût raisonnable
  - Générer les alternatives
    - heuristiques
  - Choisir la meilleure
    - estimation approximative du coût

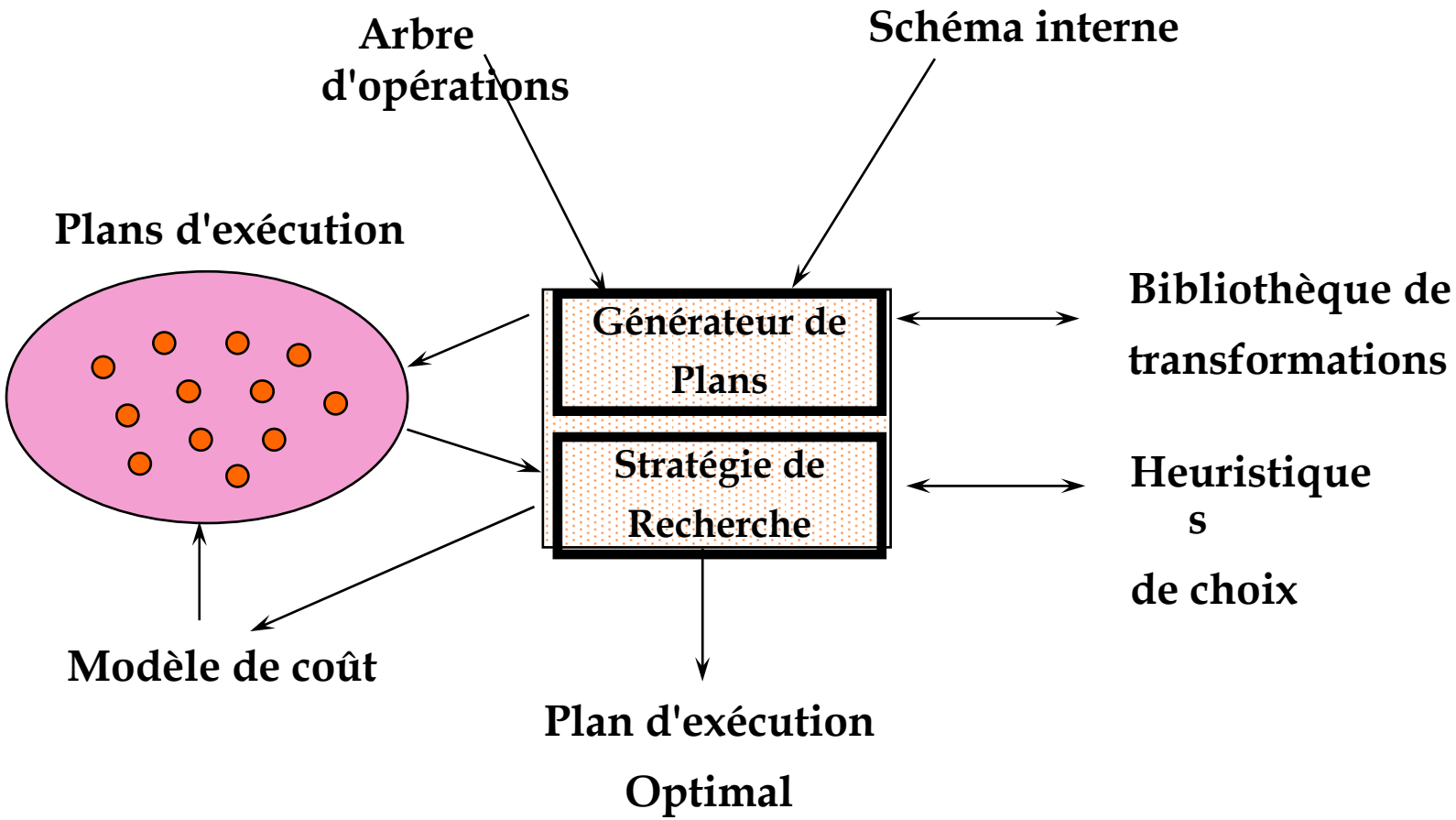
Plusieurs plans d'exécution pour un arbre algébrique

- Pour chaque opération logique
  - plusieurs choix d 'opérations physiques

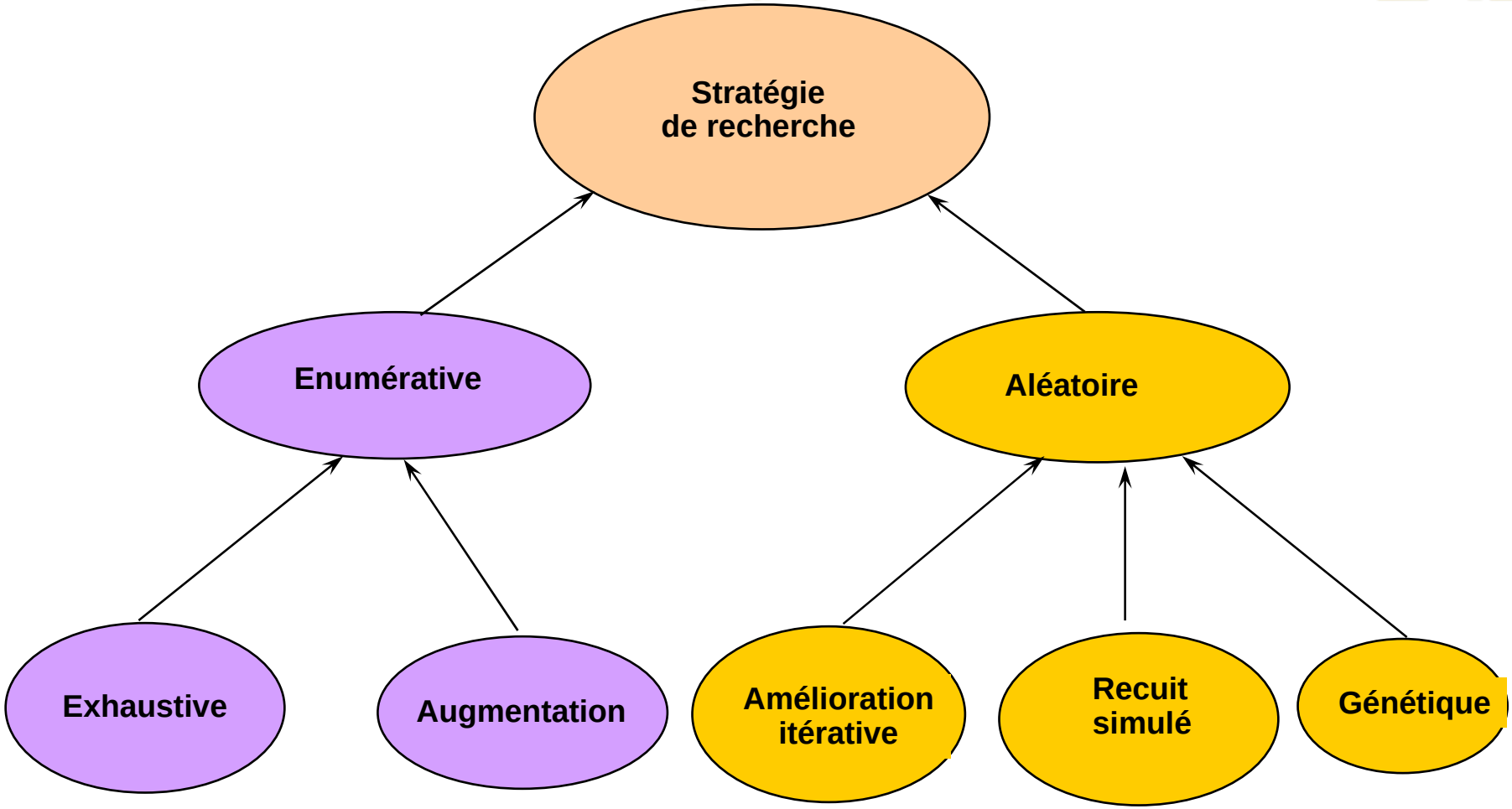


  - etc.

Choix du meilleur plan

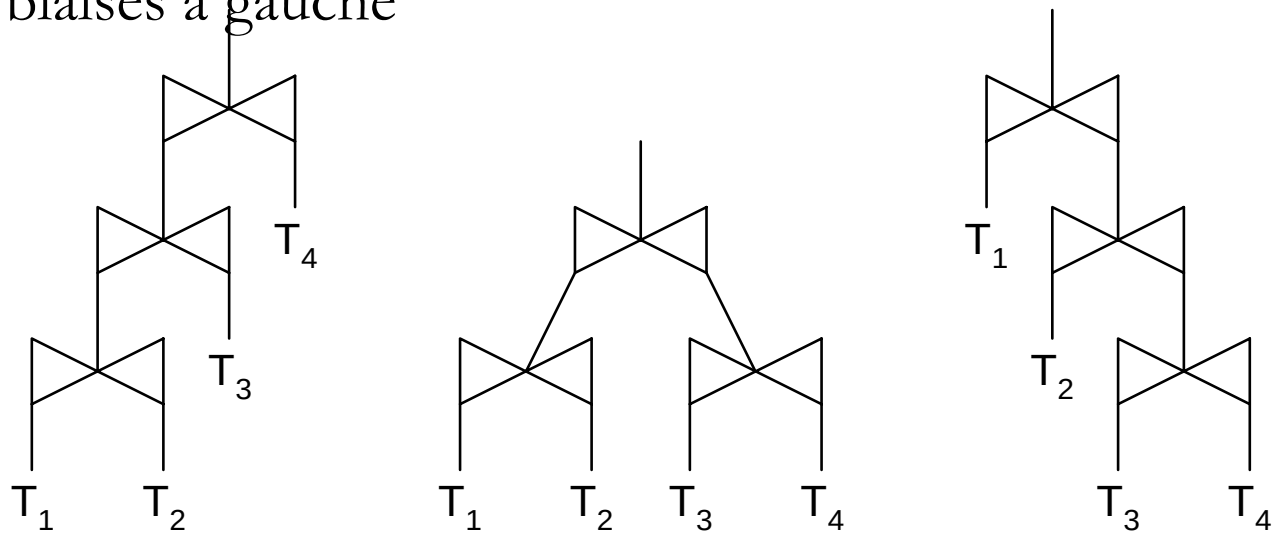


Différentes Stratégies



Heuristique : arbres biaisés à gauche seulement

- Jointure de n tables
  - (2*(n-1))!/(n-1)! ordres différents pour n tables
  - n! biaisés à gauche



Arbre biaisé à gauche Arbre équilibré Arbre biaisé à droite

Contrôle du processus d'optimisation

- Cas Oracle
  - Outils
    - EXPLAIN PLAN
    - SQL *Trace*
    - SQL *Analyse* (*Enterprise Manager Tuning Pack*)
    - Oracle EXPERT

Paramètre OPTIMIZER_MODE

- COST(statistique): minimise le coût estimé
  - besoin de statistiques (ANALYSE)
  - Mieux mais plus cher
  - ALL_ROWS
    - minimise le temps total (plutôt MERGE JOIN)
  - FIRST_ROWS
    - minimise temps réponse (plutôt NESTED LOOPS)
- RULE (heuristique)
  - appelée à disparaître ?

Exemple d 'utilisation de EXPLAIN PLAN

SQL> start utlxplan.sql *Pour créer la table plan_table*
Table created.
...
SQL> run
  1 explain plan
  2 set statement_id = 'com'
  3 for select * from commandes, lignes_de_commande
  4 where no_commande = commande_no_commande

Explained.

Suite

```
SQL> run
  1  SELECT LPAD(' ',2*(LEVEL-1))||operation||' '||options
  2    ||' '||object_name
  3      ||' '||DECODE(id, 0, 'Cost = '||position) "Query Plan"
  4  FROM plan_table
  5     START WITH id = 0 AND statement_id = 'com'
  6*  CONNECT BY PRIOR id = parent_id AND statement_id ='com'

Query Plan
--------------------------------------------
SELECT STATEMENT   Cost =
  NESTED LOOPS
    TABLE ACCESS FULL LIGNES_DE_COMMANDE
    TABLE ACCESS BY ROWID COMMANDES
      INDEX UNIQUE SCAN COMMANDE_PK
```

Consommation de ressources

Oracle SQL Analyze - SYSMAN@uqam-mtks51cp9t

File Edit View SQL TopSQL History Tools Help

Databases
- ora817i
  - Initialization Parame
  - TopSQL
  - SQL History
  - SQL002
    - Rule
    - Cost All Rows
    - Index Recomm
  - SQL003 [TopSQL]
    - Rule
    - Index Recomm
  - SQL001

SQL Statement: SQL002 - Rule Schema: GODIN

```
SELECT DISTINCT noarticle
    FROM commande, lignecommande
    WHERE commande.nocommande = lignecommande.nocommande
      AND noclient = 10
```

Explain | Compact View | Statistics

Total Fetches: 1

Statistics	Latest Fetch	Average Over Fetches
Elapsed Time (in seconds)	0.08	0.08
CPU Time (in seconds)	0.00	0.00
First Row (in seconds)	0.08	0.08
Logical Blocks Read	5	5.00
Physical Blocks Read	3	3.00
Recursive Calls	0	0.00
Database Calls	4	4.00
Chained Rows	0	0.00
Number of Rows Returned	6	6.00

For Help, press F1 OVR

Cas DB2

- Documentation
  - http://www-4.ibm.com/cgi-bin/software/db2www/library/pubs.d2w/report#UDBPUBS
- Paramètre OPTIMIZATION CLASS
  - Classe 0
    - algorithme vorace d'énumération des ordres de jointure
    - boucles imbriquées seulement
  - Classe 1
    - jointure par tri-fusion
  - Classe 2
    - optimisation de l 'accès par index secondaire par manipulation des listes de références
    - jointures en étoile
  - Classes 3
    - algorithme de programmation dynamique
    - réécritures des SELECT imbriqués en jointures
  - Classe 5
    - statistiques non uniformes (histogrammes)
  - Classe 7
    - examine plus de plans
  - Classe 9
    - considère toutes les alternatives

Optimisation basée sur modèle de coût

- Soit Q une requête à optimiser:
- Procédure:
  - Énumérer tous les plans $\{P_1, ..., P_m\}$ pour chaque requête (notons que chaque requête possède un ensemble d'opérations $O_1, ..., O_k$)
  - Pour chaque plan P_j

    Pour chaque opération O_i du plan P_j, énumérer les routines d'accès

    Sélectionner la routine ayant le coût le moins élevé

$$\textbf{Coût}(P_i) = \Sigma_{(l=1,\, k)}\ \textbf{min}(O_l)$$

$$\textbf{Coût}\ (Q): \Sigma_{(h=1,\, m)}\ \textbf{Coût}(P_h)$$

Exemple

- Une requête avec 3 opérations (une projection, deux sélections et une jointure) : P1 S1 S2 et J1
  Généralement, on exécute les sélections d'abord, ensuite la jointure et enfin la projection.

- Énumérer tous les plans:
  - Plan A: S1 S2 J1 P1 Plan B: S2 S1 J1 P1
  - Choisir un plan (commençons par Plan A)
    - Pour chaque opération, énumérer toutes les routines:

      Opération S1 : Linear Search et binary search

      Opération S2 : Linear Search et indexed search

      Opération J1 : Nested Loop join et indexed join

Exemple (suite)

- Choose the least cost access routine for each operation

  Opération S1 least cost access routine is binary search at a cost of 10 blocks

  Opération S2 least cost access routine is linear search at a cost of 20 blocks

  Opération J1 least cost access routine is indexed join at a cost of 40 blocks

- Thus the total cost for Plan A will be: 70

- Répéter le même principe pour le plan B:

  Opération S2 least cost access routine is binary search at a cost of 20 blocks

  Opération S1 least cost access routine is indexed search at a cost of 5 blocks

  Opération J1 least cost access routine is indexed join at a cost of 30 blocks

  Thus the total cost for Plan B will be: 55

  ➔ Résultat: Plan B le meilleur plan.

Hints

- Les directives ‘’hints’’ sont des moyens de forcer l’optimiseur à choisir un plan d’exécution.
- Les directives peuvent influencer les plans d’exécution et par conséquent la performance des requêtes
- Exemple
  - Sous Oracle:
    - `SELECT /*+ some hints here */`

Hints sous Oracle

Hint	Explication
CHOOSE	If statistics are available (see DBMS_STATS package) then use Cost Based otherwise use Rule Based.
RULE	Use the Rule based Optimizer (even if table statistics are available).
ALL_ROWS	Use the Cost Based optimizer to maximize throughput (minimize system resources)
FIRST_ROWS	Use the Cost Based optimizer to minimize response time

Hints sous Oracle

Hint	Explication
ORDERED	Join tables in the order in which they appear in the FROM clause of the query. Try changing around the order to see if performance improves.
FULL (table_name)	Perform a full table scan on the named table even if an index is available.
INDEX (table_name index_name)	Forces the use of the named index on the named table (rather than a full table scan).

Exemple

```
SELECT /*+ ALL_ROWS */ nom
FROM Ouvrier;

SELECT /*+ FIRST_ROWS */ nom
FROM Ouvrier;

SELECT /*+ COST */ nom, ville
FROM Ouvrier WHERE age > 25 AND Ville in ('Poitiers', 'Niort', 'Angouleme' ,
    'LaRochelle');
```

Hints: Accès aux données

- **FULL -** Force a full table-scan. Use when querying large portions or all of a table or for small static tables.

```
SELECT /*+ FULL(<tablename>) */ * FROM <tablename>;
```

- **ROWID** - Scan a table by ROWID.
- **CLUSTER** - Cluster scan on a cluster.
- **HASH -** Cluster hash scan.
- **INDEX**:

```
SELECT /*+ INDEX(<tablename indexname [indexname]>) */
  * FROM <tablename> WHERE ...;
```

- **INDEX_COMBINE** : Forces bitmap access
- **INDEX_JOIN** - Forces the use of an index join.

Index

```
SELECT /*+ RULE*/ nom
FROM Client
WHERE noClient = 10 ;
```

```
SELECT /*+ INDEX(EMPLOYÉ INDEX_SEXE)*/ nom, adresse
FROM EMPLOYÉ WHERE SEXE = ‘F’
```

Hints: Jointure

- **ORDERED** - Forces table access in a specified order from multiple selected tables

```
SELECT /*+ ORDERED */ * FROM <tablename, tablename>
```

- **USE_MERGE** - Join each table with a sort-merge join.
- **USE_HASH** - Join each table with a hash-merge join.
- **[NO] PARALLEL** - perform full table scans in parallel by splitting a query into multiple threads running simultaneously, the results are merged to be returned as a single row set.
  - Example: This query breaks the query up into four processes.

```
SELECT /*+FULL(table) PARALLEL(table, 4) */ field1, field2
FROM table
```

Optimiseur

- L'optimiseur ORACLE suit une approche classique :
  - Génération de plusieurs plans d'exécution.
  - Estimation du coût de chaque plan généré.
  - Choix du meilleur et exécution.
- Tout ceci est automatique, **mais il est possible d'influer, voire de forcer le plan d'exécution**.

Création des statistiques

- Calcul de la taille et du nombre de lignes :

  ANALYSE TABLE Film COMPUTE STATISTICS FOR TABLE

- Analyse des index :

  ANALYSE TABLE Film COMPUTE STATISTICS FOR ALL INDEX

- Analyse de la distribution des valeurs :

  ANALYSE TABLE Film COMPUTE STATISTICS FOR COLUMNS titre, genre SIZE 20

Optimisation de la Jointure

- Trois implémentations possibles
  - Boucles Imbriquées
  - Sort Merge Join
  - Hach Join
- Index de jointure [Valduriez 1984]
  - Pré-calcule la jointure
  - Maintenance de la relation de jointure entre les clés étrangères les clés primaires

Exemple

Index de jointure		
$ROWID_{ventes}$	$ROWID_{Client}$	$ROWID_{Article}$
v1	c1	a1
v2	c2	a2
v3	c3	a1
…		

Table Client		
ROWID	noClient	nomVille
c1	1	Montréal
c2	2	Ottawa
c3	3	Montréal
c4	4	Paris

Table Article		
ROWID	noArticle	catégorie
a1	10	Conifère
a2	20	Conifère
a3	40	Conifère
a4	50	Feuillu
a5	60	Feuillu

Table Vente				
ROWID	noClient	noArticle	dateVente	montant
v1	1	10	10/01/2000	100
v2	2	20	10/01/2000	200
v3	3	10	10/01/2000	500
…				

Index de Jointure et les SGBD Commerciaux

- Index de jointure binaire (bitmap join index)
  - Un index binaire défini sur une table qui référence une colonne d'une autre table.
- Syntaxe:

```
CREATE BITMAP INDEX bjx_vente_client_sexe
ON     VENTE(Client.Sexe)
FROM VENTE V, CLIENT C
WHERE V.noclient = C.noclient;
```

Exemple d'un index de jointure binaire

	Sexe ='M'	Sexe ='F'
ROWID1	1	0
ROWID2	0	1
ROWID3	1	0
ROWID4	1	0
ROWID5	1	0
ROWID6	1	0

```
SELECT COUNT(*)
FROM VENTE V, CLIENT C
    WHERE V.noclient = C.noclient
    AND Sexe = 'F';
```

Partitionnement

- Fragmentation
  - Décomposer une table en fragments
  - Approche de conception logique d'une base de données
- Différents types de fragmentation
  - Horizontale
  - Verticale
  - Mixte

Fragmentation horizontale

Décomposition de la table en groupes de lignes

Table Client(N°Client, Nom, Ville)

➔ Deux types de fragmentation horizontale:

- Primaire
- Dérivée

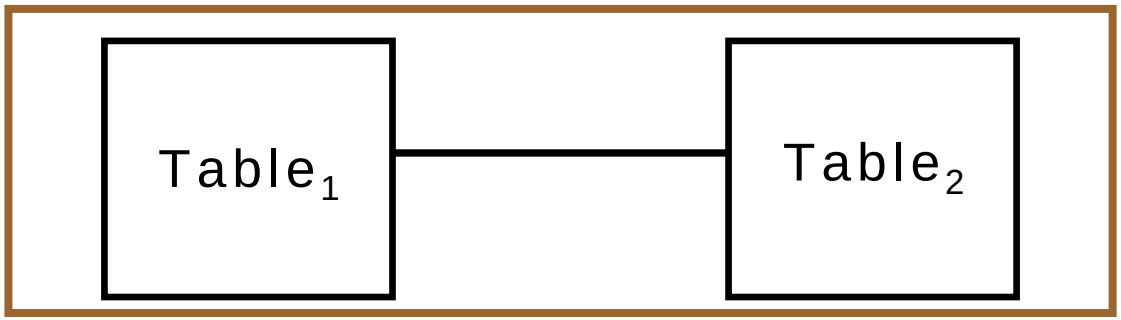
Fragmentation horizontale primaire

- Obtention des fragments horizontaux:
  - Fragmentation horizontale est définie par l'opération de sélection
- Exemple
  - Client(N°Client, Nom, Ville) peut être fragmentée :
  - Client1= SELECT * FROM Client WHERE Ville = "Paris"
  - Client2= SELECT * FROM Client WHERE Ville <> "Paris"
- Reconstruction de la relation initiale:

  Client = Client1 ∪ Client2

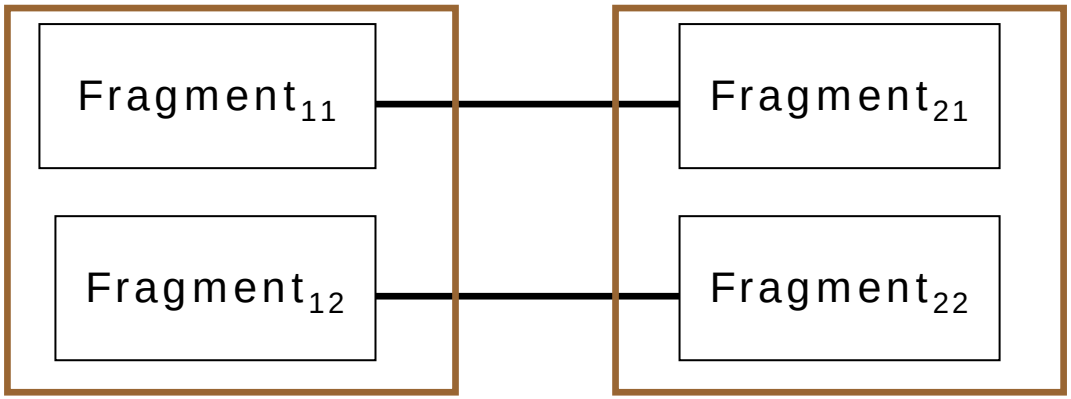
Fragmentation horizontale dérivée(I)

➔ Fragmentation d'une table en fonction des fragments horizontaux d'une autre table



Avant fragmentation

Fragments dérivés



Après fragmentation

Fragmentation horizontale dérivée(II)

- Obtention des fragments horizontaux dérivés:
  - Fragments dérivés sont obtenus par l'opération de semi-jointure
- Exemple

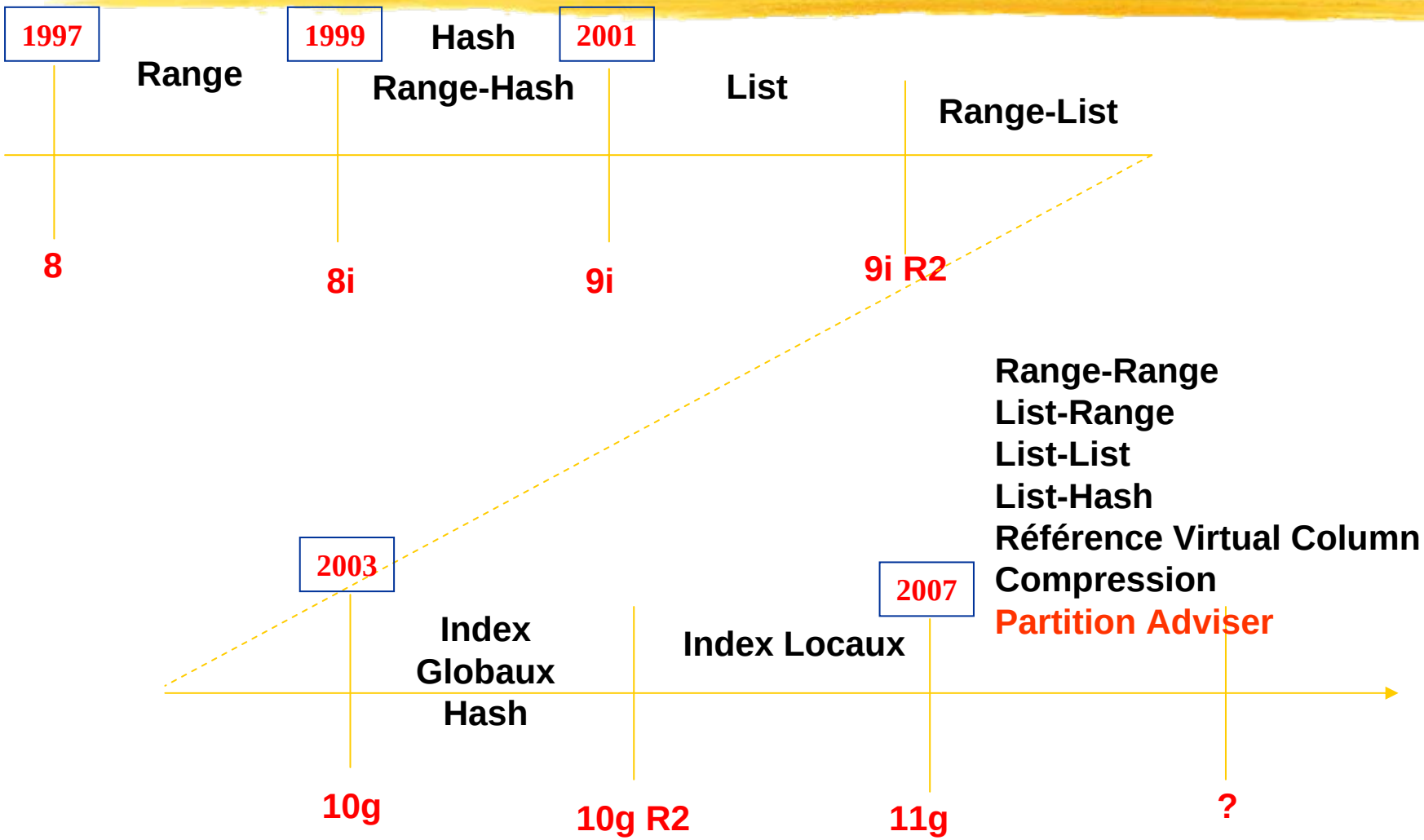
Commande(N°Client, N°Produit, Date, Qte, N°Représentant)

```
Commande1= SELECT * FROM Commande
           WHERE N°client IN
           (SELECT N°Client FROM CLIENT1)
Commande2= SELECT * FROM Commande
           WHERE N°Client IN
            (SELECT N°Client FROM CLIENT2)
```

- Reconstruction de la relation initiale:

Commande = Commande1 ∪ Commande2

Fragmentation dans ORACLE



Exemples

Modes Simples:

Range, List, Hash

```
CREATE TABLE Client_Age
(Client_id  NUMBER(5),
Nom  VARCHAR2(20),
Age NUMBER(4),
Sexe VARCHAR2(2),
Département NUMBER(2))
PARTITION BY RANGE (Age)
 (
PARTITION Client_Moins_27 VALUES LESS THAN (27) TABLESPACE TBSMoins27,
PARTITION Client_18_60 VALUES LESS THAN (61) TABLESPACE TBS27-60,
PARTITION Client_Plus_60 VALUES LESS THAN (MAXVALUES) TABLESPACE TBSPlus60
);
```

Exemple: List Partitioning

```
Create table dept_part
(deptno    number(2),
dname      varchar2(14),
loc        varchar2(13))
partition by LIST (dname)
(partition d1_east  values ('BOSTON', 'NEW YORK'),
partition d2_west  values ('SAN FRANCISCO', 'LOS ANGELES'),
partition d3_south values ('ATLANTA', 'DALLAS'),
partition d4_north values ('CHICAGO', 'DETROIT'));
```

Exemple de Hash Partitioning

```
CREATE TABLE Departement(Dname VARCHAR(15), Head
    VARCHAR(15),Faculty VARCHAR(15))
    PARTITION BY HASH (Dname)
    (PARTITION Dept1 TABLESPACE TS01,
    PARTITION Dept2 TABLESPACE TS02,
    PARTITION Dept3 TABLESPACE TS03);
```

Partitionnement Composé

```
CREATE TABLE Client_Age_Sexe
(Client_id  NUMBER(5),
Nom  Varchar2(20),
Sexe Varchar2(2),
Age Number(4),
Département NUMBER(2))
PARTITION BY RANGE (Age)
SUBPARTITION BY LIST (Sexe)
SUBPARTITION TEMPLATE(
SUBPARTITION  Client1 VALUES ('M') TABLESPACE MasculinTBS ,
SUBPARTITION Client2 VALUES ('F') TABLESPACE FéminiinTBS )
(
PARTITION Client_Moins_27 VALUES LESS THAN (27),
PARTITION Client_27_60 VALUES LESS THAN (61),
PARTITION Client_Plus_60 VALUES LESS THAN (MAXVALUES)
);
```

Fragmentation verticale(I)

Décomposition de la table en groupes de colonnes

Table Client(N°Client, Nom, Sexe, Ville)

N°Client	Nom

Fragment vertical$_1$

N°Client	Sexe	Ville

Fragment vertical$_2$

Fragmentation verticale(II)

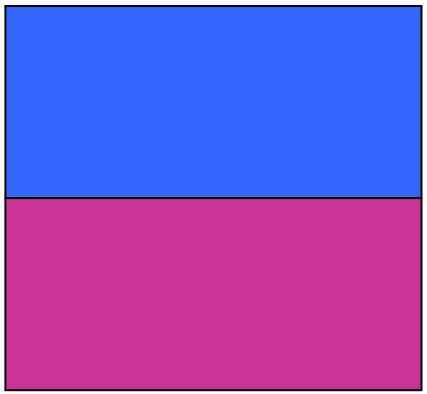
- Obtention des fragments verticaux:
  - Fragmentation verticale est définie par l'opération de projection
- Exemple
  - Client(N°Client, Nom, Sexe,Ville) peut être fragmentée :
  - $Client_1$= SELECT N°Client, Nom FROM Client
  - $Client_2$= SELECT N°Client, Sexe, Ville FROM Client
- Reconstruction de la relation initiale:

  Client = $Client_1$ join $Client_2$

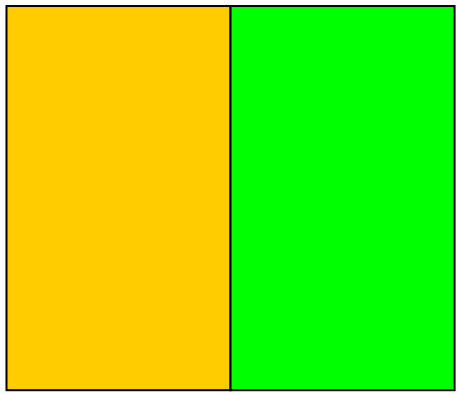
Pourquoi le N°Client est dupliqué dans les deux fragments?

Fragmentation mixte

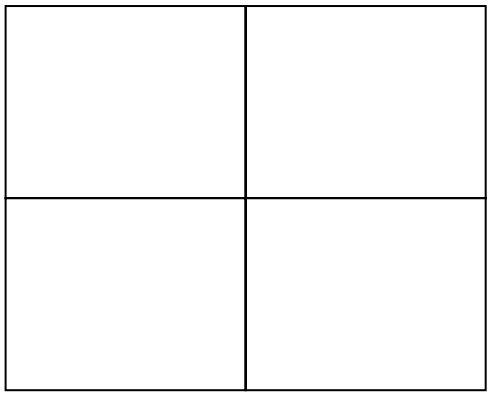
- Obtention des fragments mixtes:
  - Fragmentation mixte résulte de l'application successive d'opérations de fragmentation horizontale et de fragmentation verticale



Horizontale



Verticale

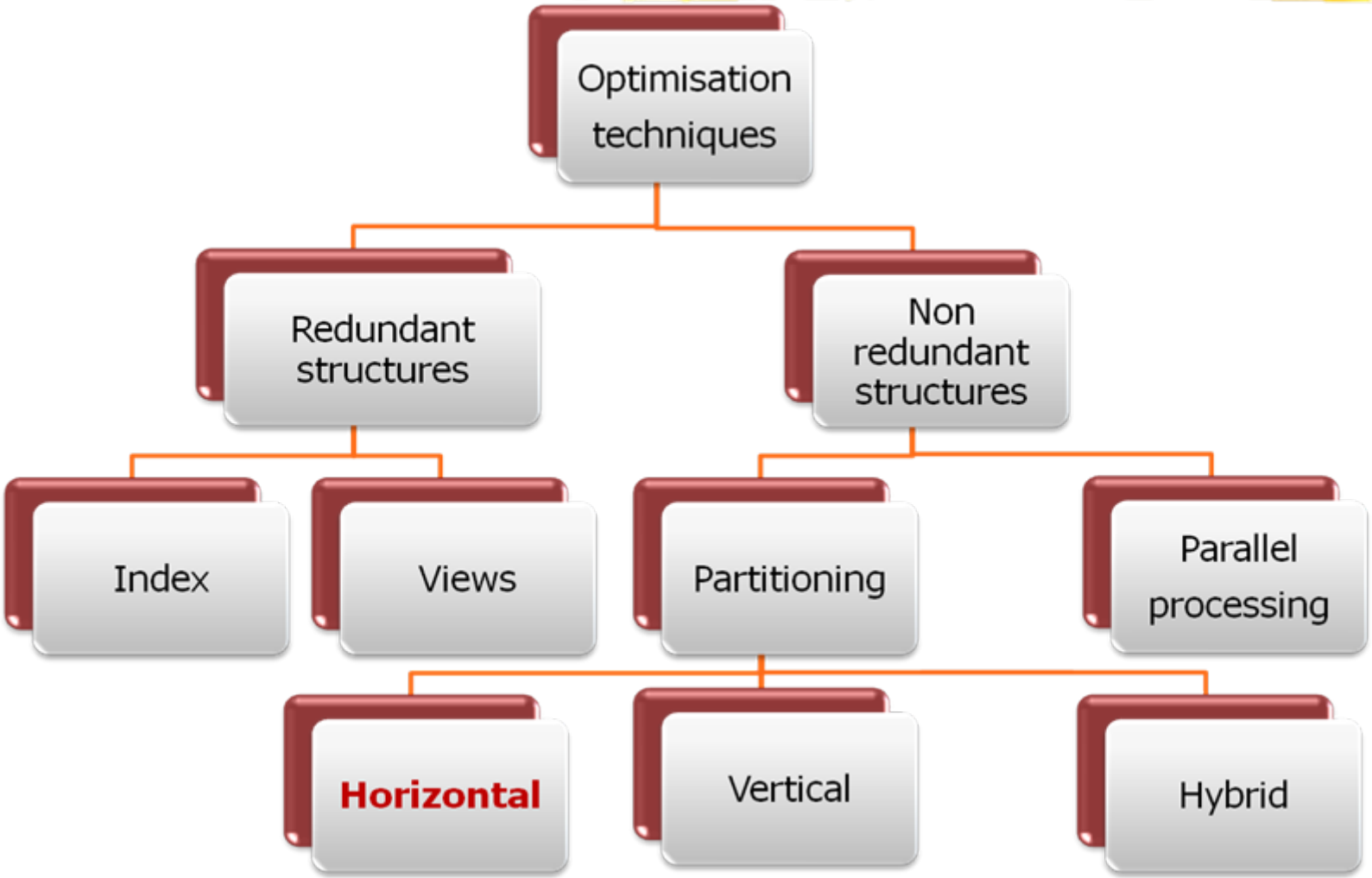


Mixte

Avantages et inconvénients de la fragmentation

+ Réduction des accès non pertinents
+ Parallélisme intra-requête
+ Combinée avec d'autres techniques d'optimisation (index, etc.)
– génération des fragments disjoints est un problème difficile
– Accès multiples aux fragments nécessitent des opérations de jointure et d'union
– La migration des données (conséquence d'une mauvaise fragmentation horizontale)

Ce qu'il faut retenir…



Indexation

- Index Simples
  - Index définis sur une seule table quelque soit leur nature (hash, bitmap, B-tree)
- Index Multi-tables
  - Index de Jointure
  - Index de jointure en étoile

➲ **Utilisation avec Modération**

➲ **Espace de stockage**

➲ **Maintenance**

Fragmentation

- Fragmentation horizontale:
  - Supportée dans tous les SGBD commerciaux.
- Fragmentation verticale: pas encore...
- Modes simples (fragmentation horizontale)
  - Range, List, Hash.
- Modes Composés:
  - Toute combinaison des modes simples (Oracle 11g).
- Attention:
  - Fragmentation sur un nombre d'attributs > 2 n'est pas supportée
  - Fragmentation dérivée supportée sur une seule table.

Problème de la fragmentation dérivée

- Le fait que le schéma d'une base de données est totalement connecté, on risque d'avoir plusieurs possibilités de fragmentation dérivée [Bellatreche, Entity-Relationship'98]
- Lorsqu'une relation a plusieurs parents (relation père/fils)

Comment choisir?

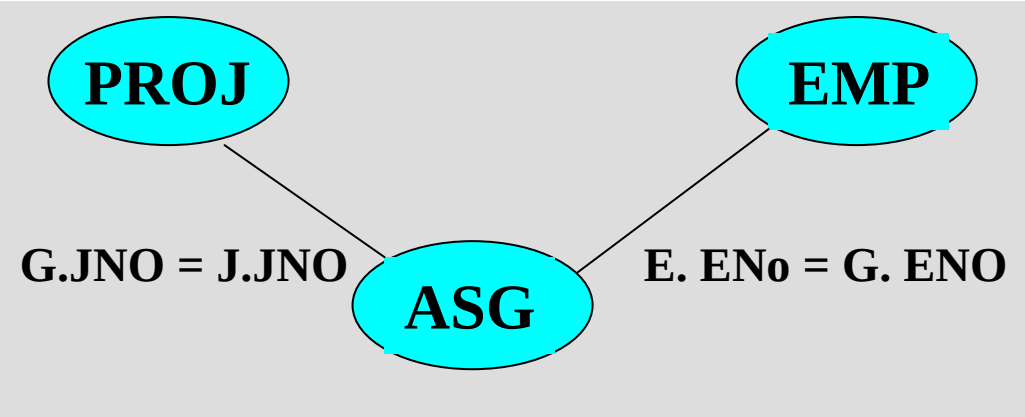
- Les caractéristiques de la jointure:
  - Choisir la solution qui optimise les opérations de jointure
- La performance d'une opération de jointure dans le contexte répartie dépend de la taille de fragments (ou relations) manipulés.

Jointure distribuée

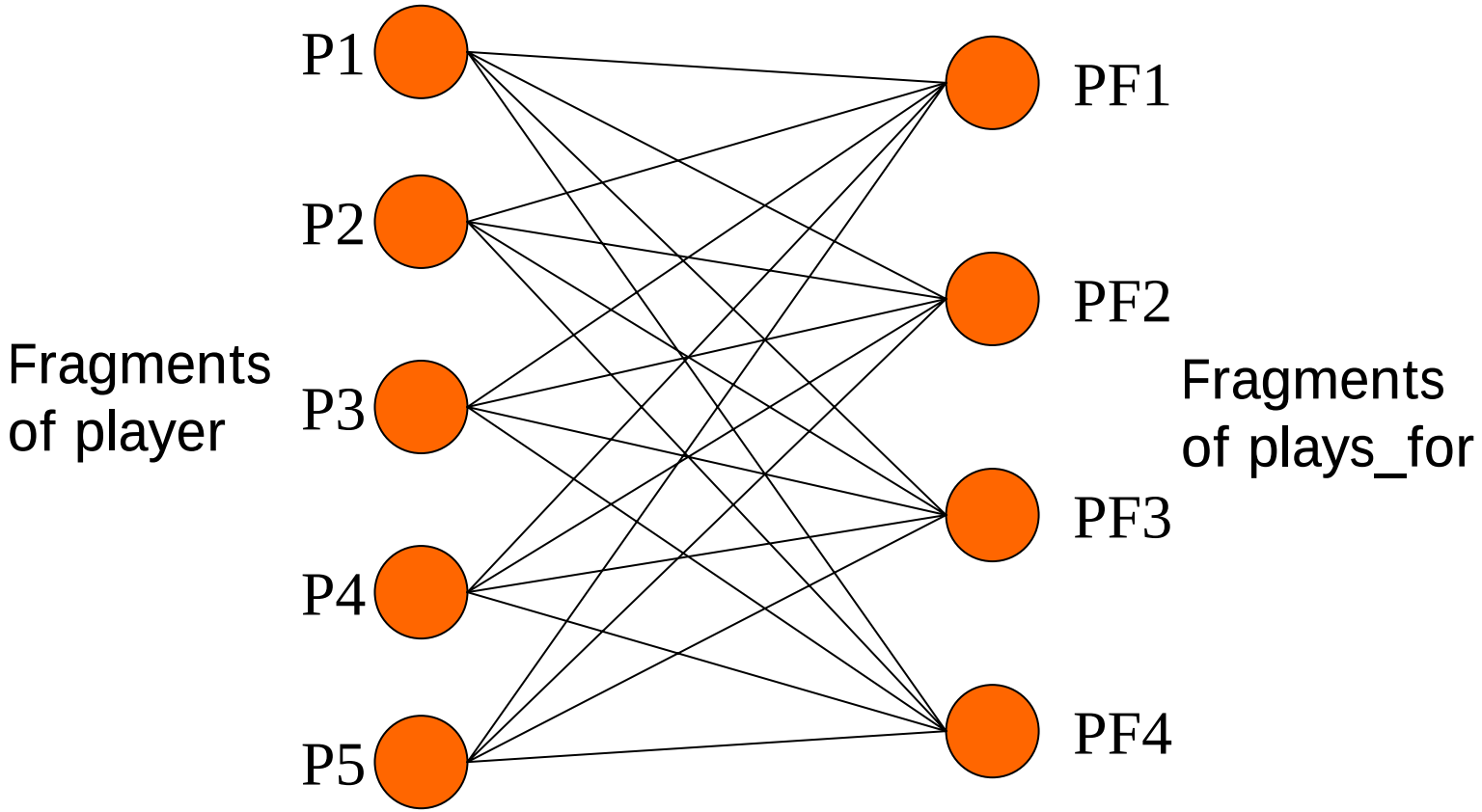
- Une jointure répartie est une jointure entre des relations horizontalement fragmentées
- Une jointure répartie peut être représentée par un graphe appelé graphe de jointure:
  - Les nœuds representent les fragments des tables partitionnées,
  - Une arête est introduite lorsque la jointure entre les fragments correspondants n'est pas vide

Graphe de jointure

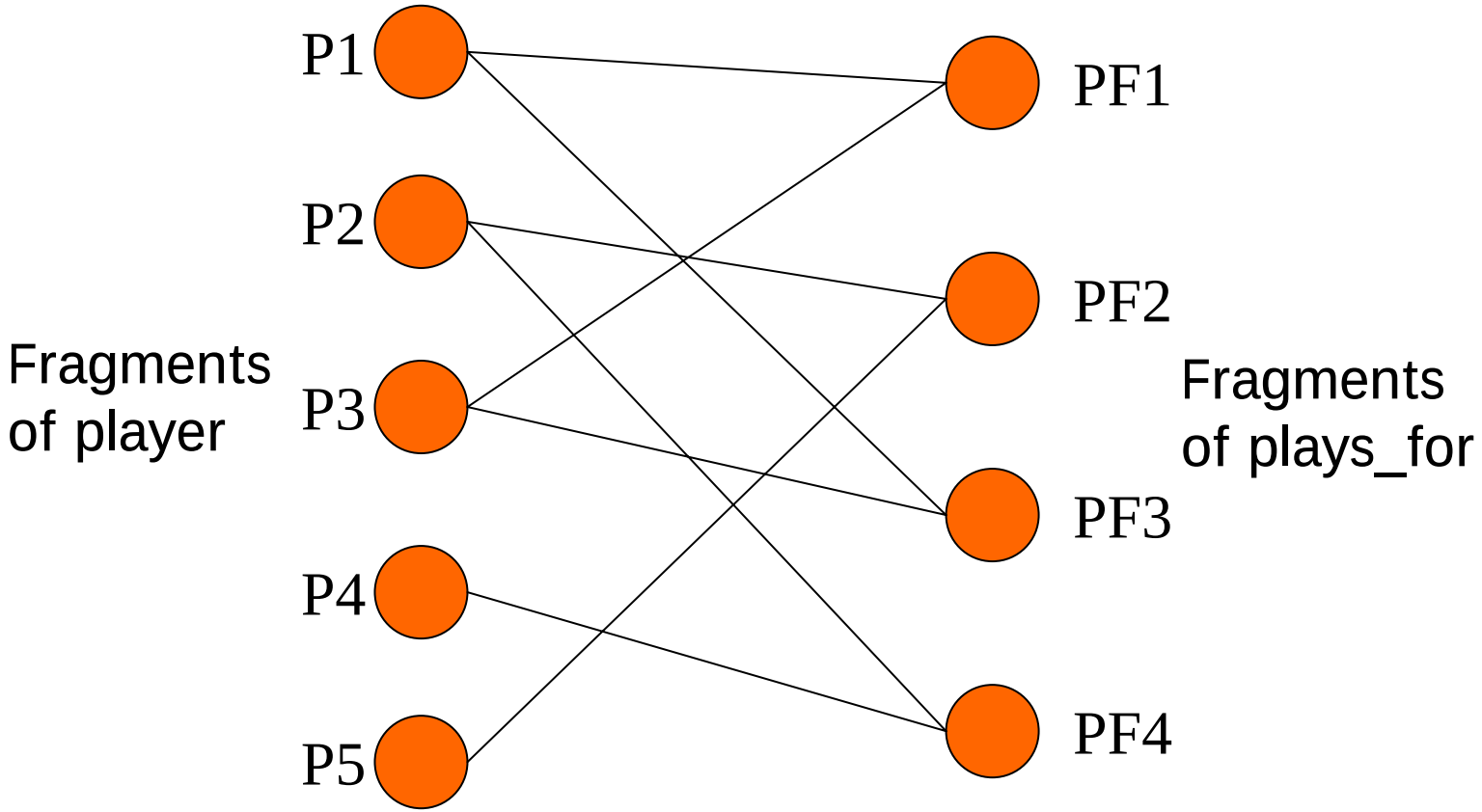
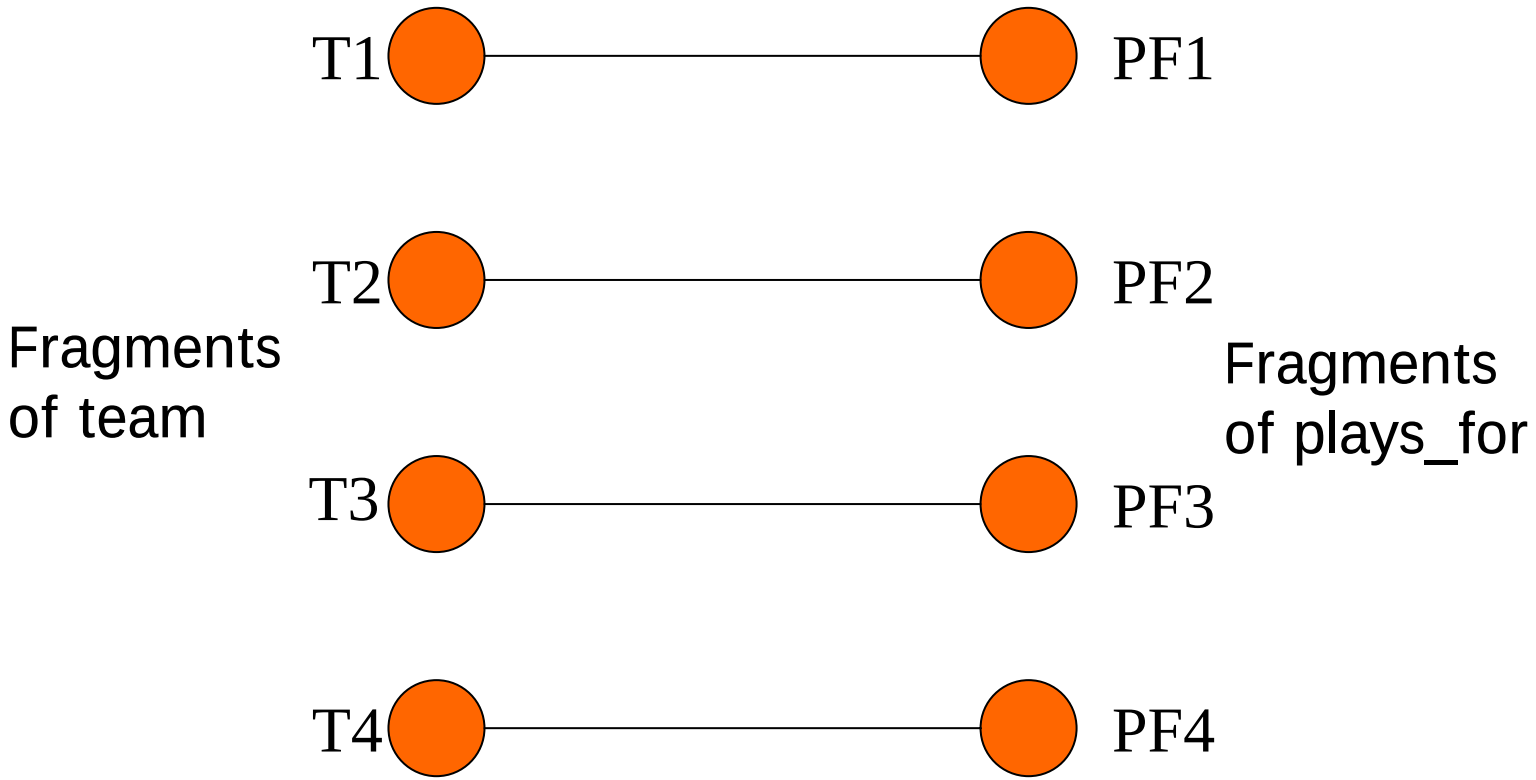
```
SELECT Ename, Resp
FROM E, G, J
WHERE E. ENo = G. ENO
AND G.JNO = J.JNO
AND JNAME = ``CAD''
AND DUR >= 36
AND Title = ``Prog''
```



Graphe de jointure total



Graphe de jointure partiel



Graphe de jointure simple